



Subject Name: Programming for Problem Solving
Subject Code: BE01R00121

CHAPTER NO- 3: CONTROL STRUCTURE IN C		
TOPIC:1 Decision Making Statements		
SHORT QUESTIONS		
1	Define nested-if-else statement. [NLJIET]	2
	Nested "if else statements" simply means the use of conditional statement if..else inside another conditional statement if..else. <u>The basic format of the Nested if-else statement is:</u> Syntax: <pre> if(test_expression one) {//Statement block Executes when the test expression one is true. if(test_expression two) { //Statement block Executes when the test expression two is true. } else { //else statement block- Statement block Executes when the test expression two is false } } else { //else statement block - Statement block Executes when the test expression one is false } </pre>	
2	What is ternary operator? [NLJIET]	1



	Ternary operator: We use the ternary operator in C to run one code when the condition is true and another code when the condition is false. Syntax of Ternary Operator (Conditional operator) The syntax of ternary operator is : <pre>testCondition ? expression1 : expression 2;</pre> The <code>testCondition</code> is a boolean expression that results in either true or false. If the condition is • true - expression1 (before the colon) is executed • false - expression2 (after the colon) is executed The ternary operator takes 3 operands (<code>condition</code>, <code>expression1</code> and <code>expression2</code>). Hence, the name ternary operator.	
3	What is the output of following code: [NLJIET] <pre>void main() { int i=5; switch(i) { case 3: printf(" three"); case 4: printf(" four"); case 5: printf(" five"); case 6: printf(" six"); break; case 7: printf(" seven"); default: printf(" default"); } }</pre>	2
	Output: five six Explanation: The break statement is not mentioned in switch case so it runs till the first break	



	statement is countered.	
	DESCRIPTIVE QUESTIONS	
1	Explain switch case in 'C' with the help of an example. (Jun-2017-OLD) [NLJIET] Describe switch case statement with example. (Jun-2019-OLD) [NLJIET] Explain <i>switch</i> statement with an example. (Mar-2023-NEW) [NLJIET]	3 4 4
	<u>Switch statement:</u> In C, the switch case statement is used for executing one condition from multiple conditions. It is similar to an if-else-if ladder. The switch statement consists of conditional-based cases and a default case. Syntax of switch Statement in C <pre> switch(expression) { case value1: statement_1; break; case value2: statement_2; break; . . . case value_n: statement_n; break; default: default_statement; } </pre> Exploring Emerging Technologies Rules for switch statement in C language 1) The <i>switch expression</i> must be of an integer or character type. 2) The <i>case value</i> must be an integer or character constant.	

3) The *case value* can be used only inside the switch statement.

4) The *break statement* in switch case is not must. It is optional. If there is no break statement found in the case, all the cases will be executed present after the matched case. It is known as *fall through* the state of C switch statement.

Break in switch case

This keyword is used to stop the execution inside a switch block. It helps to terminate the switch block and break out of it. When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.

The **break statement is optional**. If omitted, execution will continue on into the next case. The flow of control will fall through to subsequent cases until a break is reached.

Default in switch case

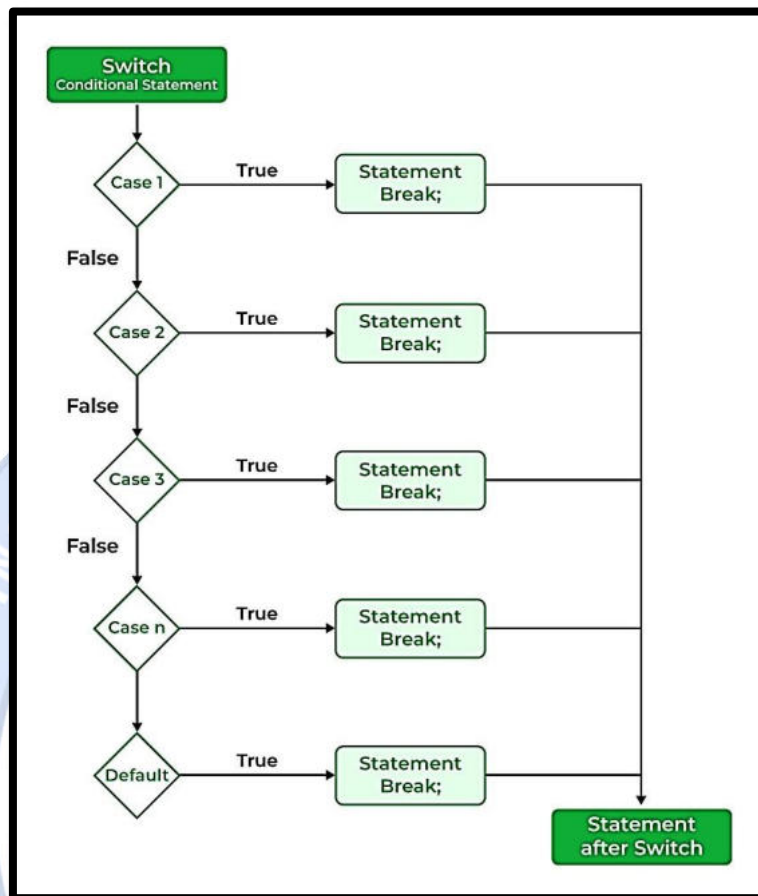
The default keyword is used to specify the set of **statements to execute if there is no case match**.

It is **optional** to use the default keyword in a switch case. Even if the switch case statement does not have a default statement, it would run without any problem.

Flowchart of switch statement:



Exploring Emerging Technologies



Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
// switch variable
```

```
int var = 1;
```

```
// switch statement
```

```
switch (var) {
```

```
case 1:
```

```
printf("Case 1 is Matched.");
```

```
break;
```

```
case 2:
```

```
printf("Case 2 is Matched.");
break;
```

```
case 3:
    printf("Case 3 is Matched.");
    break;
```

```
default:
    printf("Default case is Matched.");
    break;
```

```
}

return 0;
}
```

Output

Case 1 is Matched.

How switch Statement Work?

The working of the switch statement in C is as follows:

1. **Step 1:** The switch variable is evaluated.
2. **Step 2:** The evaluated value is matched against all the present cases.
3. **Step 3A:** If the matching case value is found, the associated code is executed.
4. **Step 3B:** If the matching code is not found, then the default case is executed if present.
5. **Step 4A:** If the break keyword is present in the case, then program control breaks out of the switch statement.
6. **Step 4B:** If the break keyword is not present, then all the cases after the matching case are executed.
7. **Step 5:** Statements after the switch statement are executed.

2	Explain nested if else ladder with suitable example. (Mar-2022-NEW) [NLJIET]	4
---	---	---

Nested "if else statements" simply means the use of conditional statement if..else inside another conditional statement if..else.

The basic format of the Nested if-else statement is:

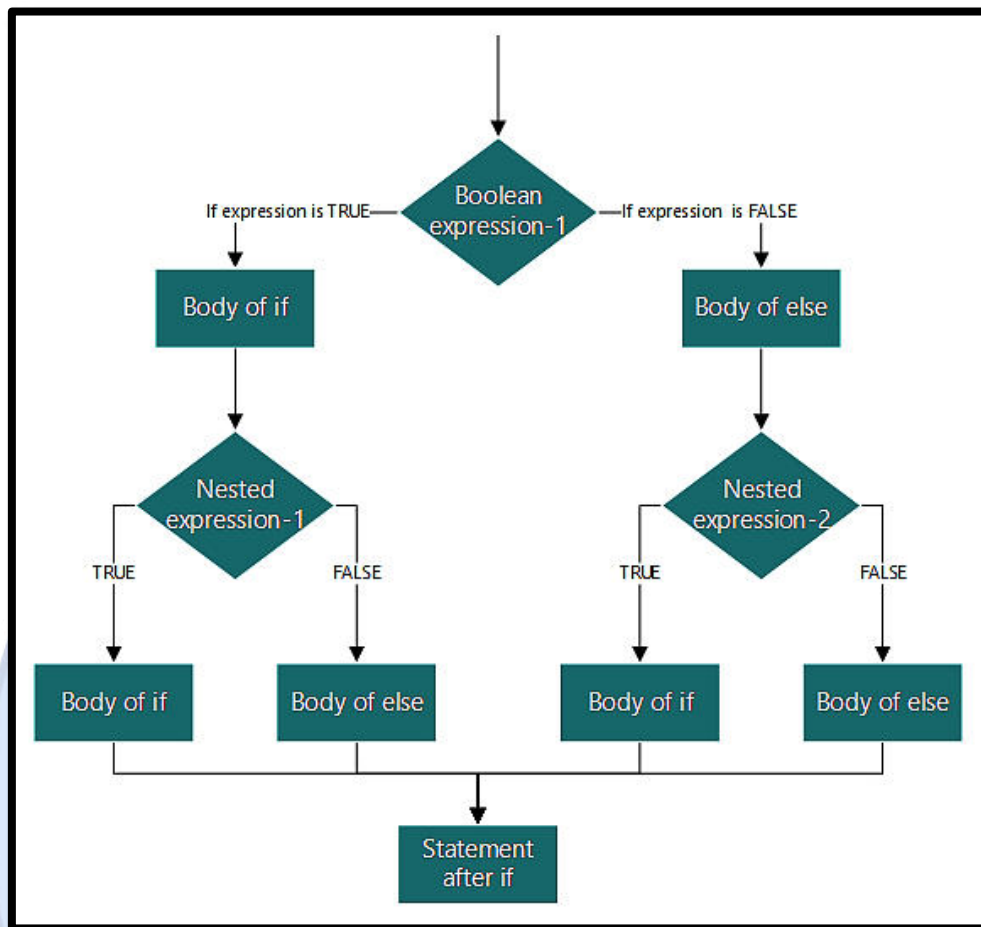
Syntax:

```
if(test_expression one)
{//Statement block Executes when the test expression one is true.
    if(test_expression two) {
        //Statement block Executes when the test expression two is true.
    }
    else
    {
        //else statement block- Statement block Executes when the test expression two
is false
    }
}
else
{
    //else statement block - Statement block Executes when the test expression one
is false
}
```

Flowchart of nested if..else:



Exploring Emerging Technologies



Example:

program to select and print the largest of the three nos. using nested-if-else statement

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main() {
```

```
    int n1, n2, n3;
```

```
    printf("Enter three numbers: ");
```

```
    scanf("%d %d %d", &n1, &n2, &n3);
```

```
    // outer if statement
```



```

if (n1 >= n2) {

    // inner if...else
    if (n1 >= n3)
        printf("%d is the largest number.", n1);
    else
        printf("%d is the largest number.", n3);
}

// outer else statement
else {

    // inner if...else
    if (n2 >= n3)
        printf("%d is the largest number.", n2);
    else
        printf("%d is the largest number.", n3);
}

return 0;
}

```

Output:

Enter three numbers: 2

4

6

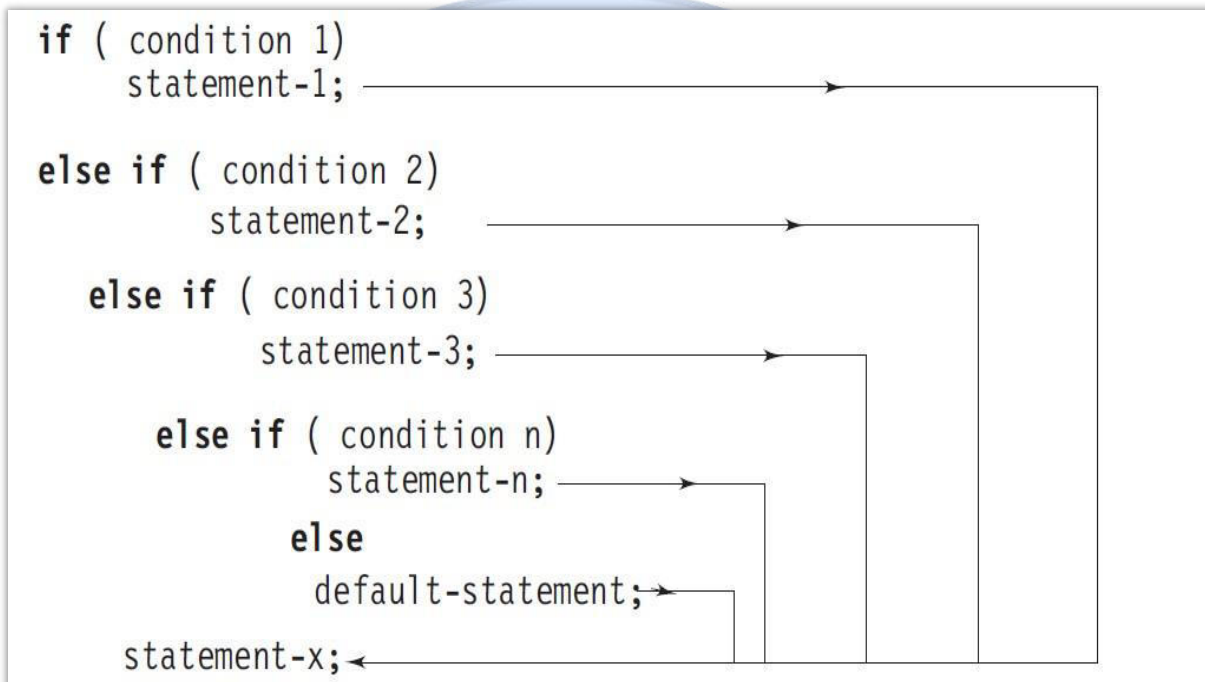
6 is the largest number.

3 Explain else-if ladder with proper example.(Mar-2023-OLD)[NLJIET]

7

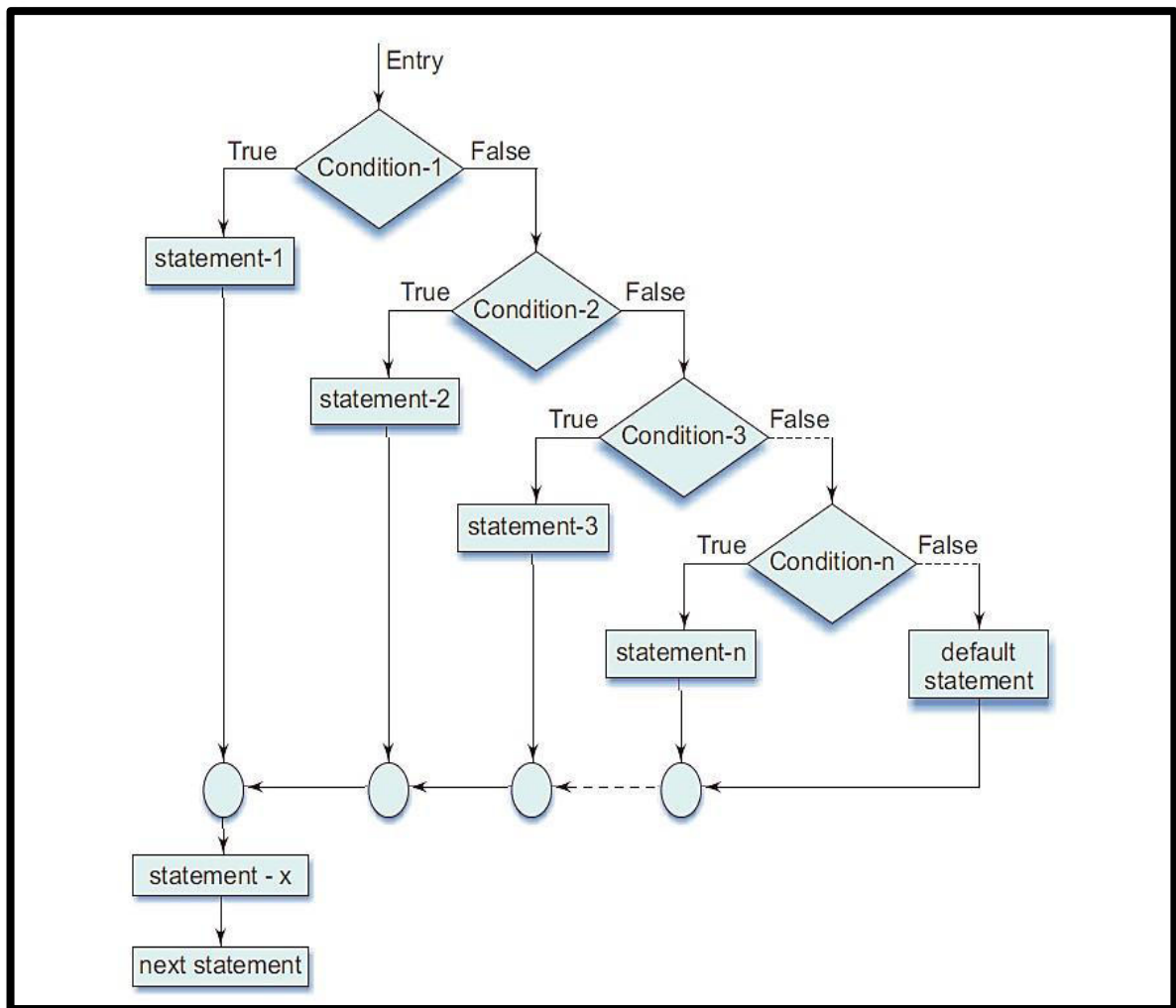
THE ELSE IF LADDER

There is another way of putting ifs together when multipath decisions are involved. A multipath decision is a **chain of ifs in which the statement associated with each else is an if. It takes the following general form**



- This construct is known as the else if ladder. The conditions are evaluated from the top (of the ladder), downwards.
- As soon as a true condition is found, the statement associated with it is executed and the control is transferred to the statement-x (skipping the rest of the ladder).
- When all the n conditions become false, then the final else containing the default-statement will be executed
- The figure below shows the logic of execution of else if ladder statements.

Exploring Emerging Technologies



```
// C Program to check whether  
// a number is positive, negative  
// or 0 using if else if ladder  
#include <stdio.h>
```

```
int main()  
{  
    int n = 0;  
  
    // all Positive numbers will make this  
    // condition true  
    if (n > 0) {
```



```

    printf("Positive");
}

// all Negative numbers will make this
// condition true
else if (n < 0) {
    printf("Negative");
}

// if a number is neither Positive nor Negative
else {
    printf("Zero");
}
return 0;
}

```

Output

Zero

PROGRAMS

- | | | |
|---|---|---|
| 1 | Write a program to select and print the largest of the three nos. using nested-if-else statement. (Jun-2014-OLD) (Jan-2019-OLD)[NLJIET] | 7 |
| | Write a program to select and print the largest of the three nos. using nested-if-else statement. (Dec-2015-OLD) [NLJIET] | 4 |

```

#include <stdio.h>
#include <conio.h>

int main() {

    int n1, n2, n3;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &n1, &n2, &n3);

    // outer if statement

```

```

if (n1 >= n2) {

    // inner if...else
    if (n1 >= n3)
        printf("%d is the largest number.", n1);
    else
        printf("%d is the largest number.", n3);
}

// outer else statement
else {

    // inner if...else
    if (n2 >= n3)
        printf("%d is the largest number.", n2);
    else
        printf("%d is the largest number.", n3);
}

return 0;
}

```

Output:

Enter three numbers: 2

4

6

6 is the largest number.

2

Write a menu driven C program for simple calculator. Also draw flowchart. (Jun-2016-OLD) [NLJIET]

7

Develop a simple program to Add, subtract and multiply two numbers using switch statement. (Jan-2017-OLD) (Jan-2019-OLD) [NLJIET]

7

Write a program to perform addition, multiplication, subtraction and division with switch statement. (Jun-2014-OLD) [NLJIET]

7

Program Solution-1:

```
#include<stdio.h>
#include<conio.h>
int main()
{
    float a,b;
    int op;

    printf("Enter the values of a & b: ");
    scanf("%f %f",&a,&b);
    printf(" Menu: \n 1.Addition\n 2.Subtraction\n 3.Multiplication\n
4.Division\n");
    printf("Enter your Choice : ");
    scanf("%d",&op);
    switch(op)
    {
        case 1 :
            printf("Sum of %f and %f is : %f",a,b,a+b);
            break;
        case 2 :
            printf("Difference of %f and %f is : %f",a,b,a-b);
            break;
        case 3 :
            printf("Multiplication of %f and %f is : %f",a,b,a*b);
            break;
        case 4 :
            printf("Division of Two Numbers is %f : ",a/b);
            break;
        default:
            printf(" Enter Your Correct Choice.");
            break;
    }
    return 0;
```



```
}
```

Output:

Enter the values of a & b: 1

3

Menu:

- 1.Addition
- 2.Subtraction
- 3.Multiplication
- 4.Division

Enter your Choice : 3

Multiplication of 1.000000 and 3.000000 is : 3.000000

Or

Program Solution-2:

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int main() {
```

```
    char op;
```

```
    double first, second;
```

```
    printf("Enter an operator (+, -, *, /): ");
```

```
    scanf("%c",&op);
```

```
    printf("Enter two operands: ");
```

```
    scanf("%lf%lf", &first, &second);
```

```
    switch (op)
```



```
{
    case '+':
        printf("%.1lf + %.1lf = %.1lf", first, second, first + second);
        break;
    case '-':
        printf("%.1lf - %.1lf = %.1lf", first, second, first - second);
        break;
    case '*':
        printf("%.1lf * %.1lf = %.1lf", first, second, first * second);
        break;
    case '/':
        printf("%.1lf / %.1lf = %.1lf", first, second, first / second);
        break;
    // operator doesn't match any case constant
    default:
        printf("Error! operator is not correct");
}

return 0;
}
```

Output:

Enter an operator (+, -, *, /): +

Enter two operands: 3

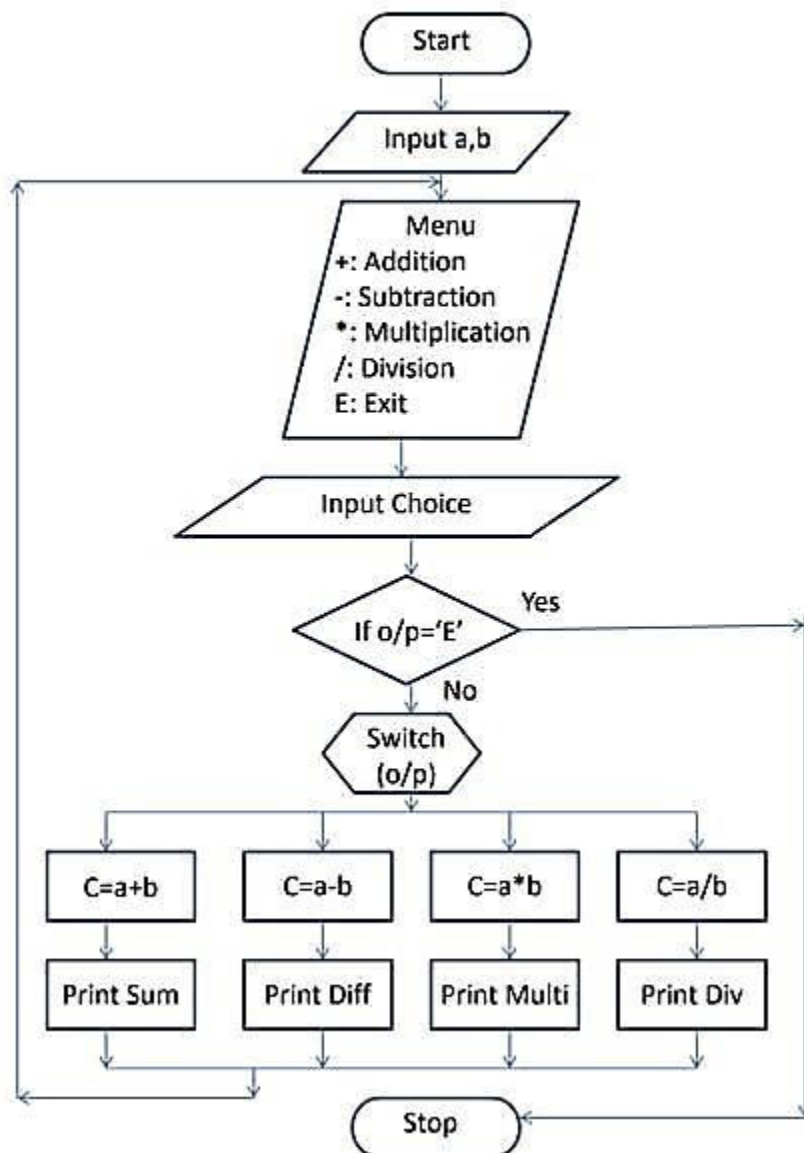
6

3.0 + 6.0 = 9.0

Flowchart:

Exploring Emerging Technologies

Flow Chart



3

Write a program to accept start number and end number from the user and print all the numbers in the range. (Jun-2017-OLD) [NLJIET]

7

Program:

```
#include<stdio.h>
#include<conio.h>
```



```
int main() {

    int s,e,i;
    printf("Enter starting Range: ");
    scanf("%d",&s);
    printf("Enter ending Range: ");
    scanf("%d",&e);
    printf("\n The numbers in given range between %d and %d are: \n",s,e);
    for(i=s;i<=e;i++)
    {
        printf("\t %d",i);
    }
    return 0;
}
```

Output:

Enter starting Range: 6

Enter ending Range: 16

The numbers in given range between 6 and 16 are:

6 7 8 9 10 11 12 13 14 15 16

4	Explain switch case statement with example to read number between 1 to 7 and print relatively day Sunday to Saturday. (Jan-2019-NEW) [NLJIET]	7
	<u>Switch statement:</u> In C, the switch case statement is used for executing one condition from multiple conditions. It is similar to an if-else-if ladder. The switch statement consists of conditional-based cases and a default case. Syntax of switch Statement in C	

```
switch(expression)
{
case value1: statement_1;
    break;
case value2: statement_2;
    break;
.
.
.
case value_n: statement_n;
    break;

default: default_statement;
}
```

Rules for switch statement in C language

- 1) The *switch expression* must be of an integer or character type.
- 2) The *case value* must be an integer or character constant.
- 3) The *case value* can be used only inside the switch statement.
- 4) The *break statement* in switch case is not must. It is optional. If there is no break statement found in the case, all the cases will be executed present after the matched case. It is known as *fall through* the state of C switch statement.

Break in switch case

This keyword is used to stop the execution inside a switch block. It helps to terminate the switch block and break out of it. When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.

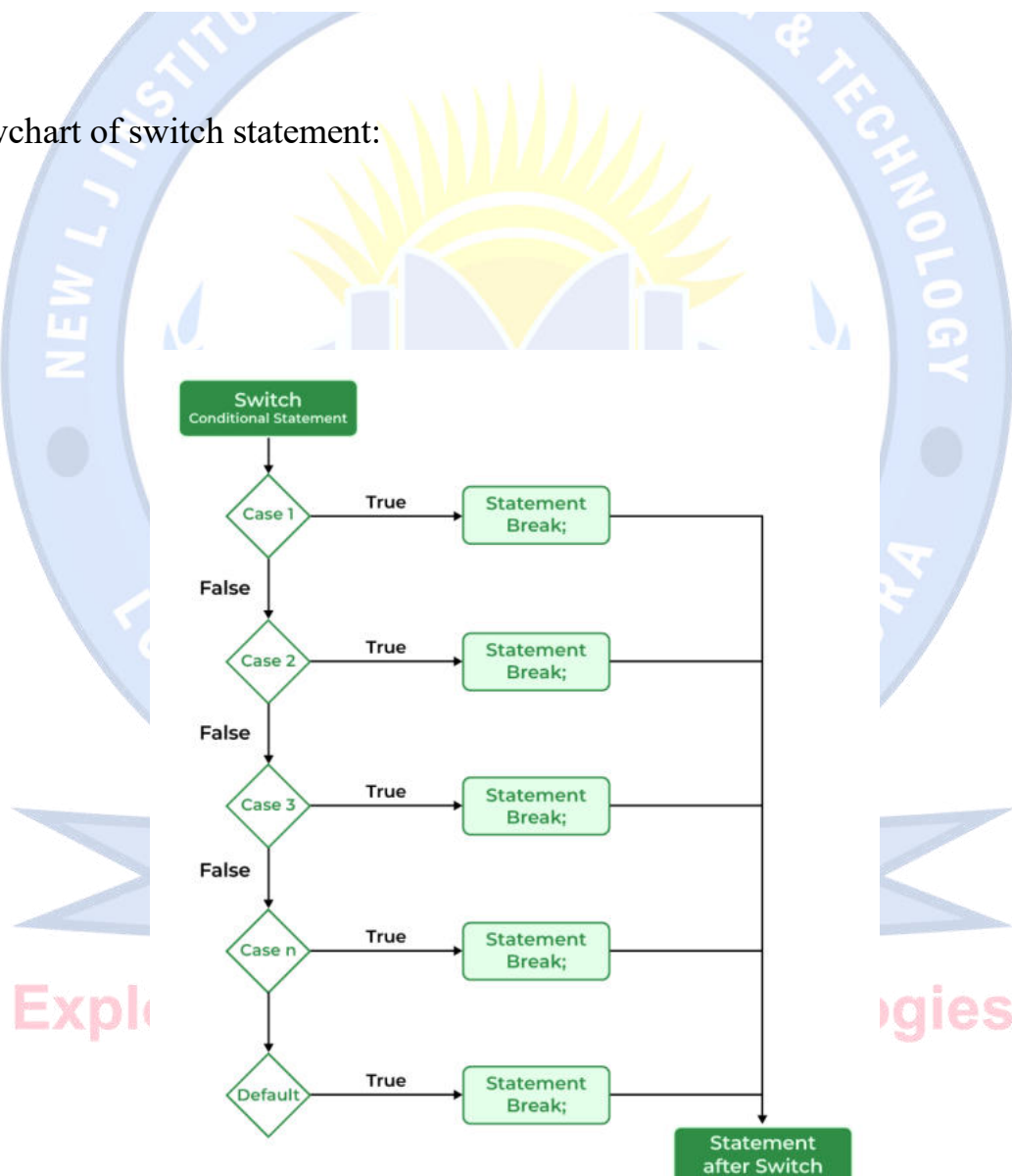
The **break statement is optional**. If omitted, execution will continue on into the next case. The flow of control will fall through to subsequent cases until a break is reached.

Default in switch case

The default keyword is used to specify the set of **statements to execute if there is no case match**.

It is **optional** to use the default keyword in a switch case. Even if the switch case statement does not have a default statement, it would run without any problem.

Flowchart of switch statement:



Example:



```
#include<stdio.h>
#include<conio.h>

int main() {
    int no;
    printf("\n Enter Day no between 1-7 : ");
    scanf("%d", & no);
    switch (no) {
        case 1:
            printf("\n Sunday");
            break;
        case 2:
            printf("\n Monday");
            break;
        case 3:
            printf("\n Tuesday");
            break;
        case 4:
            printf("\n Wednesday");
            break;
        case 5:
            printf("\n Thursday");
            break;
        case 6:
            printf("\n Friday");
            break;
        case 7:
            printf("\n Saturday");
            break;
        default:
            printf("\n Please Enter Proper Input");
            break;
    }
    return 0;
}
```

**Output:**

Enter Day no between 1-7 : 7

Saturday

5

Write a C program to read numbers 1 to 7 and print relatively day Sunday to Saturday using switch statement. (NOV-2020-NEW) [NLJIET]

7**Program:**

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int main() {
```

```
    int no;
```

```
    printf("\n Enter Day no between 1-7 : ");
```

```
    scanf("%d", &no);
```

```
    switch (no) {
```

```
    case 1:
```

```
        printf("\n Sunday");
```

```
        break;
```

```
    case 2:
```

```
        printf("\n Monday");
```

```
        break;
```

```
    case 3:
```

```
        printf("\n Tuesday");
```

```
        break;
```

```
    case 4:
```

```
        printf("\n Wednesday");
```

```
        break;
```

```
    case 5:
```

```
        printf("\n Thursday");
```




```

break;
case 6:
    printf("\n Friday");
    break;
case 7:
    printf("\n Saturday");
    break;
default:
    printf("\n Please Enter Proper Input");
    break;
}
return 0;
}

```

Output:

Enter Day no between 1-7 : 7

Saturday

6 Demonstrate a C program to input an integer number and check last digit of number is even or odd. (Jan-2019-NEW) (Jan-2020-NEW) [NLJIET]

7

Program:

```

#include<stdio.h>
#include<conio.h>

```

```

void main() {
    int no,last_dig;
    printf("\n Enter no : ");
    scanf("%d",&no);
    last_dig=no%10;
    if(last_dig%2==0)
    {
        printf("\n the last digit %d is even",last_dig);
    }
}

```



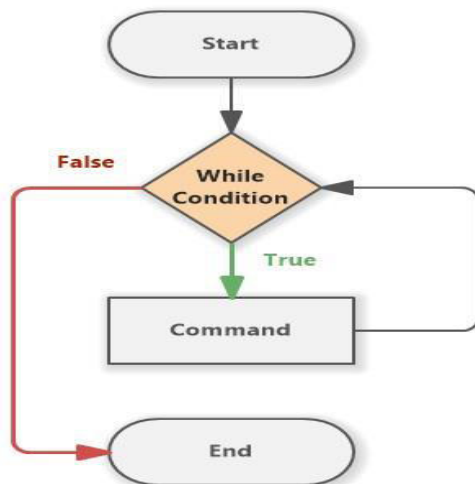
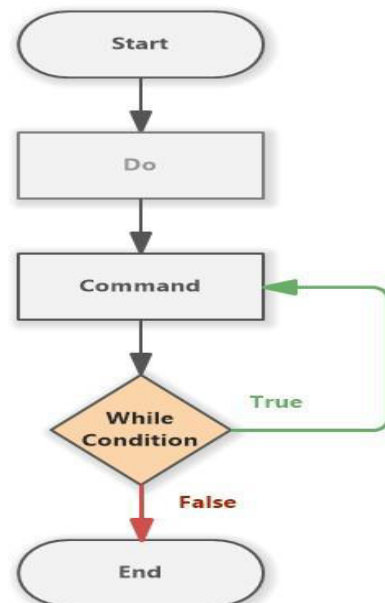
	<pre> } else { printf("\n the last digit %d is odd",last_dig); } } </pre> Output: Enter no : 1236 the last digit 6 is even	
7	Write code to find out largest of 2 numbers using ternary operator. (Jun-2017-OLD) [NLJIET]	4
	Program: <pre> #include<stdio.h> #include<conio.h> void main() { int a,b; printf("\n Enter no 1 : "); scanf("%d",&a); printf("\n Enter no 2 : "); scanf("%d",&b); (a>b) ? printf("\n %d is largest",a) : printf("\n %d is largest",b); //use or ternary operator } </pre> Output: Enter no 1 : 3	



	Enter no 2 : 6	
	6 is largest	
	TOPIC:2 Looping Statements, Nesting of Control Structures	
	SHORT QUESTIONS	
3	Why nesting of loop is required? [NLJIET]	2
	• A nested loop is simply a loop inside other loop . • It is useful when we are dealing with more number of iterations • Reduces the number of lines of program • reusing of code is possible • memory size usage will be reduced.	
6	How many times will the given loop run? [NLJIET] for(i=0; i>5; i++).	2
	Zero times – Loop won't get executed. Because condition is false for i=0 as 0 is not greater than 5.	
7	What is the output of the following code. [NLJIET] void main() { int i; for(i=1; i<=5; i++) printf(" %d ", i/2); }	2
	Output: 0 1 1 2 2 As fraction i/2 is printed in integer datatype.	
8	How many times following loop will be executed. [NLJIET] main() { int i = 32766; while(i<= 32767) { printf("%d\n",i); i = i+ 1; } }	2



	Loop will be executed Twice. 1) $i = 32766$, condition $i \leq 32767$ is true, o/p: 32766, $i++$ gives $i = 32767$ 2) $i = 32767$, condition $i \leq 32767$ is true, o/p: 32766 32767, $i++$ gives $i = 32768$ 3) $i = 32768$, condition $i \leq 32767$ is false. Loop terminates.																	
9	What is the difference between while and do while loop?(Mar-2023-OLD)[NLJIET]	1																
2	Compare while and do-while loop. [NLJIET]	2																
	<table><thead><tr><th>WHILE</th><th>DO-WHILE</th></tr></thead><tbody><tr><td>Condition is checked first then statement(s) is executed.</td><td>Statement(s) is executed atleast once, thereafter condition is checked.</td></tr><tr><td>It might occur statement(s) is executed zero times, If condition is false.</td><td>At least once the statement(s) is executed.</td></tr><tr><td>No semicolon at the end of while. while(condition)</td><td>Semicolon at the end of while. while(condition);</td></tr><tr><td>If there is a single statement, brackets are not required.</td><td>Brackets are always required.</td></tr><tr><td>Variable in condition is initialized before the execution of loop.</td><td>variable may be initialized before or within the loop.</td></tr><tr><td>while loop is entry controlled loop.</td><td>do-while loop is exit controlled loop.</td></tr><tr><td>while(condition) { statement(s); }</td><td>do { statement(s); } while(condition);</td></tr></tbody></table>	WHILE	DO-WHILE	Condition is checked first then statement(s) is executed.	Statement(s) is executed atleast once, thereafter condition is checked.	It might occur statement(s) is executed zero times, If condition is false.	At least once the statement(s) is executed.	No semicolon at the end of while. while(condition)	Semicolon at the end of while. while(condition);	If there is a single statement, brackets are not required.	Brackets are always required.	Variable in condition is initialized before the execution of loop.	variable may be initialized before or within the loop.	while loop is entry controlled loop.	do-while loop is exit controlled loop.	while(condition) { statement(s); }	do { statement(s); } while(condition);	
WHILE	DO-WHILE																	
Condition is checked first then statement(s) is executed.	Statement(s) is executed atleast once, thereafter condition is checked.																	
It might occur statement(s) is executed zero times, If condition is false.	At least once the statement(s) is executed.																	
No semicolon at the end of while. while(condition)	Semicolon at the end of while. while(condition);																	
If there is a single statement, brackets are not required.	Brackets are always required.																	
Variable in condition is initialized before the execution of loop.	variable may be initialized before or within the loop.																	
while loop is entry controlled loop.	do-while loop is exit controlled loop.																	
while(condition) { statement(s); }	do { statement(s); } while(condition);																	

WHILE**DO-WHILE****DESCRIPTIVE QUESTIONS**

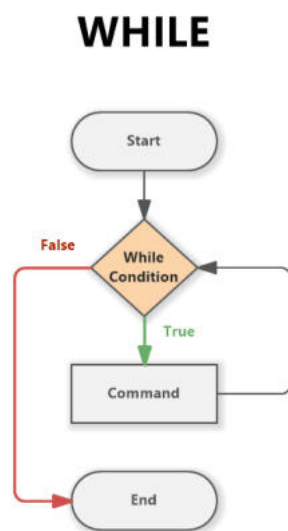
1	List the various types of loop available in C and explain their syntax. (Dec-2013-OLD)[NLJIET]	7
	Explain various types of loop available in C with example. (Jun-2014-OLD)[NLJIET]	7
	What do you understand by looping? Explain different types of loops in C with example and compare them. (Jan-2015-OLD) [NLJIET]	7
	Explain different loops used in C language. (Jun-2015-OLD) [NLJIET]	7
	Explain various looping control structures with suitable example. (Jun-2019-NEW) [NLJIET]	7
	Explain various types of loop available in C with example. (Dec-2015-OLD) [NLJIET]	4
3	Explain entry control loop and exit control loop with example. (Jun-2016-OLD)[NLJIET]	4
	Explain entry controlled and exit controlled loop with proper syntax. (Nov-2020-NEW)[NLJIET]	3
1	Give syntax of for loop. [NLJIET]	1
4	Give syntax of while loop. [NLJIET]	1



5	Give syntax of do-while loop. [NLJIET]	1
	Types of Loops in C Depending upon the position of a control statement in a program, looping statement in C is classified into two types: 1. Entry controlled loop 2. Exit controlled loop In an entry control loop in C, a condition is checked before executing the body of a loop. It is also called as a pre-checking loop. e.g while, for In an exit controlled loop, a condition is checked after executing the body of a loop. It is also called as a post-checking loop. e.g do...while ‘C’ programming language provides us with three types of loop constructs: 1. The while loop 2. The do-while loop 3. The for loop While Loop in C A while loop is the most straightforward looping structure. While loop syntax in C programming language is as follows: Syntax of While Loop in C: <pre>while (condition) { statements; }</pre> It is an entry-controlled loop. In while loop, a condition is evaluated before processing a body of the loop. If a condition is true then and only then the body of a loop is executed. After the body of a loop is executed then control again goes back at the beginning, and the condition is checked if it is true, the same	

process is executed until the condition becomes false. Once the condition becomes false, the control goes out of the loop.

Flowchart of while loop:



Following program illustrates while loop in C programming example:

```
#include<stdio.h>
#include<conio.h>
int main()
{
    int num=1;    //initializing the variable
    while(num<=10)    //while loop with condition
    {
        printf("%d\n",num);
        num++;    //incrementing operation
    }
    return 0;
}
```

Output:

1
2

3
4
5
6
7
8
9
10

Do-While loop in C

A do...while loop in C is similar to the while loop except that the condition is always executed after the body of a loop. It is also called an exit-controlled loop.

Syntax of do while loop in C programming language is as follows:

Syntax of Do-While Loop in C:

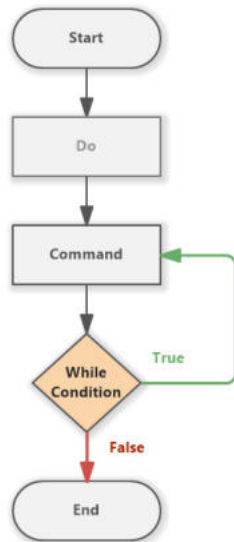
```
do {  
    statements  
} while (expression);
```

In the do-while loop, the body of a loop is always executed at least once. After the body is executed, then it checks the condition. If the condition is true, then it will again execute the body of a loop otherwise control is transferred out of the loop.

Similar to the while loop, once the control goes out of the loop the statements which are immediately after the loop is executed.

Flowchart of do...while

DO-WHILE



The following loop program in C illustrates the working of a do-while loop:

Below is a do-while loop in C example to print a table of number 2:

```
#include<stdio.h>
#include<conio.h>
int main()
{
    int num=1; //initializing the variable
    do //do-while loop
    {
        printf("%d\n",2*num);
        num++; //incrementing operation
    }while(num<=10);
    return 0;
}
```

Output:

2
4
6
8

10
12
14
16
18
20

For loop in C

A for loop is a more efficient loop structure in 'C' programming. The general structure of for loop syntax in C is as follows:

Syntax of For Loop in C:

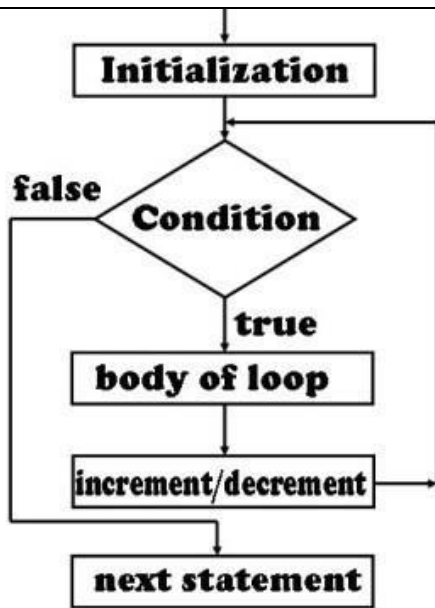
```
for (initial value; condition; incrementation or decrementation )  
{  
    statements;  
}
```

- The initial value of the for loop is performed only once.
- The condition is a Boolean expression that tests and compares the counter to a fixed value after each iteration, stopping the for loop when false is returned.
- The incrementation/decrementation increases (or decreases) the counter by a set value.

Flowchart of for loop:



Exploring Emerging Technologies



Following program illustrates the for loop in C programming example:

```
#include<stdio.h>
int main()
{
    int number;
    for(number=1;number<=10;number++) //for loop to print 1-10 numbers
    {
        printf("%d\n",number); //to print the number
    }
    return 0;
}
```

Output:

1
2
3
4
5
6
7

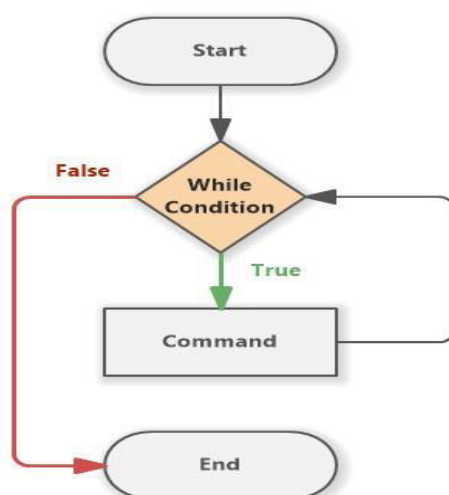
Exploring Emerging Technologies

	8 9 10 In C, the for loop can have multiple expressions separated by commas in each part. For example: <pre>for (x = 0, y = num; x < y; i++, y--) { statements; }</pre>	
2	Compare and contrast while and do while loop with example.(Mar-2022-NEW) [NLJIET] Explain while loop and do-while loop with example.(Aug-2022-NEW)[NLJIET] Differentiate <i>while</i> and <i>do..while</i> loop.(Mar-2023-NEW)[NLJIET]	4 4 4

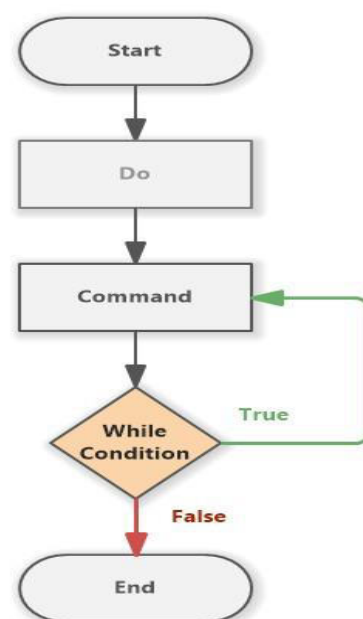
Exploring Emerging Technologies

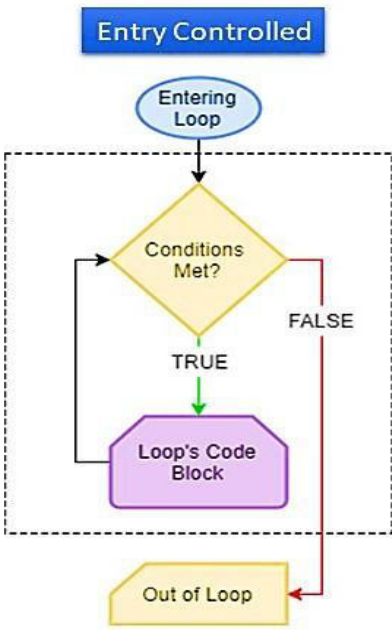
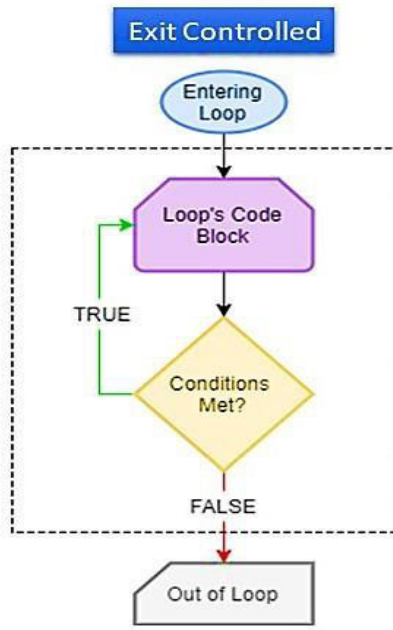
WHILE	DO-WHILE
Condition is checked first then statement(s) is executed.	Statement(s) is executed atleast once, thereafter condition is checked.
It might occur statement(s) is executed zero times, If condition is false.	At least once the statement(s) is executed.
No semicolon at the end of while. <code>while(condition)</code>	Semicolon at the end of while. <code>while(condition);</code>
If there is a single statement, brackets are not required.	Brackets are always required.
Variable in condition is initialized before the execution of loop.	variable may be initialized before or within the loop.
while loop is entry controlled loop.	do-while loop is exit controlled loop.
<code>while(condition)</code> <code>{ statement(s); }</code>	<code>do { statement(s); }</code> <code>while(condition);</code>

WHILE



DO-WHILE



3	State the difference between entry control loop and exit control loop. (Jan-2020-OLD)[NLJIET]			4										
		<table><tr><th>Entry Control Loop</th><th>Exit Control Loop</th></tr><tr><td>Entry control loop checks condition first and then body of the loop will be executed.</td><td>The exit control loop first executes the body of the loop and checks condition at last.</td></tr><tr><td>The body of the loop may or may not be executed at all.</td><td>The body of the loop will be executed at least once because the condition is checked at last</td></tr><tr><td>It is also called Top tested Loop.</td><td>It is also called Bottom tested Loop.</td></tr><tr><td>for, while are an example of an entry control loop</td><td>Do...while is an example of an exit control loop.</td></tr></table>	Entry Control Loop	Exit Control Loop	Entry control loop checks condition first and then body of the loop will be executed.	The exit control loop first executes the body of the loop and checks condition at last.	The body of the loop may or may not be executed at all.	The body of the loop will be executed at least once because the condition is checked at last	It is also called Top tested Loop.	It is also called Bottom tested Loop.	for, while are an example of an entry control loop	Do...while is an example of an exit control loop.		
Entry Control Loop	Exit Control Loop													
Entry control loop checks condition first and then body of the loop will be executed.	The exit control loop first executes the body of the loop and checks condition at last.													
The body of the loop may or may not be executed at all.	The body of the loop will be executed at least once because the condition is checked at last													
It is also called Top tested Loop.	It is also called Bottom tested Loop.													
for, while are an example of an entry control loop	Do...while is an example of an exit control loop.													
	Entry Controlled  Exit Controlled 													
4	List down three constructs for performing loop operations in C language. Write general form of same. (Jan-2017-OLD) [NLJIET]			3										

'C' programming language provides us with three types of loop constructs:

1. The while loop
2. The do-while loop
3. The for loop

While Loop in C

A while loop is the most straightforward looping structure. It is an entry-controlled loop.

While loop syntax in C programming language is as follows:

```
while (condition) {  
    statements;  
}
```

Do-While loop in C

A do...while loop in C is similar to the while loop except that the condition is always executed after the body of a loop. It is also called an exit-controlled loop.

Syntax of do while loop in C programming language is as follows:

```
do {  
    statements;  
} while (expression);
```

For loop in C

A for loop is a more efficient loop structure in 'C' programming. It is an entry controlled loop.

The general structure of for loop syntax in C is as follows:

```
for (initial value; condition; incrementation or decrementation )  
{  
    statements;
```


Do...while loop is an example of bottom tested loop.

Syntax of do while loop in C programming language is as follows:

```
do {  
    statements;  
} while (expression);
```

In the do-while loop, the body of a loop is always executed at least once. After the body is executed, it checks the condition. If the condition is true, then it will again execute the body of a loop otherwise control is transferred out of the loop.

Below is a do-while loop in C example to print a table of number 2:

```
#include<stdio.h>  
#include<conio.h>  
int main()  
{  
    int num=1;    //initializing the variable  
    do    //do-while loop  
    {  
        printf("%d\n",2*num);  
        num++;    //incrementing operation  
    }while(num<=10); //condition  
    return 0;  
}
```

Output:

2
4
6
8
10
12
14
16
18

Exploring Emerging Technologies

20

The program works as below.

1. First, we have initialized a variable 'num' with value 1. Then we have written a do-while loop.
2. In a loop, we have a print function that will print the series by multiplying the value of num with 2. Then it will check the condition $\text{num} \leq 10$.
3. After each increment, the value of num will increase by 1, and it will be printed on the screen.
4. Initially, the value of num is 1. In a body of a loop, the print function will be executed in this way: $2 * \text{num}$ where $\text{num} = 1$, then $2 * 1 = 2$ hence the value two will be printed. This will go on until the value of num becomes 10 till $2 * 10 = 20$. After that loop will be terminated and a statement which is immediately after the loop will be executed. In this case return 0.

6 Explain while loop with example. (Jan-2019-NEW) [NLJIET]

7

While Loop in C

A while loop is the most straightforward looping structure. While loop syntax in C programming language is as follows:

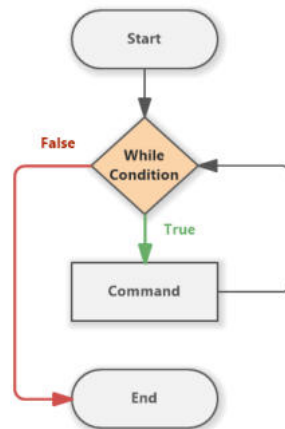
Syntax of While Loop in C:

```
while (condition) {
    statements;
}
```

It is an entry-controlled loop. In while loop, a condition is evaluated before processing a body of the loop. If a condition is true then and only then the body of a loop is executed. After the body of a loop is executed then control again goes back at the beginning, and the condition is checked if it is true, the same process is executed until the condition becomes false. Once the condition becomes false, the control goes out of the loop.

Flowchart of while loop:

WHILE



Following program illustrates while loop in C programming example:

```
#include<stdio.h>
#include<conio.h>
int main()
{
    int num=1;    //initializing the variable
    while(num<=10)    //while loop with condition
    {
        printf("%d\n",num);
        num++;    //incrementing operation
    }
    return 0;
}
```

Output:

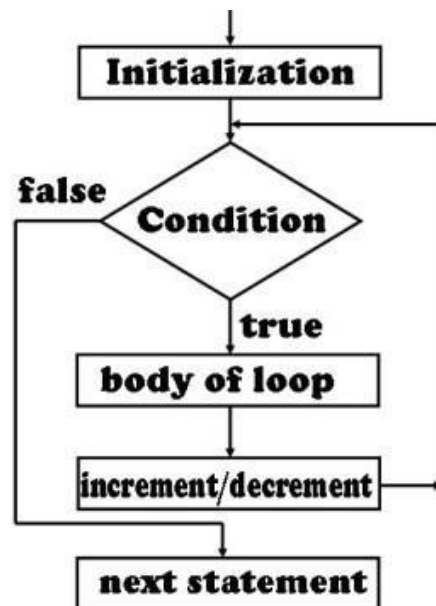
1
2
3
4



	5 6 7 8 9 10 <u>Working of program:</u> 1. We have initialized a variable called num with value 1. We are going to print from 1 to 10 hence the variable is initialized with value 1. 2. In a while loop, we have provided a condition ($\text{num} \leq 10$), which means the loop will execute the body until the value of num becomes 10. After that, the loop will be terminated, and control will fall outside the loop. 3. In the body of a loop, we have a print function to print our number and an increment operation to increment the value per execution of a loop. An initial value of num is 1, after the execution, it will become 2, and during the next execution, it will become 3. This process will continue until the value becomes 10 and then it will print the series on console and terminate the loop when num becomes 11 and condition $\text{num} \leq 10$ becomes false.	
7	Explain for loop with example. (Jan-2019-NEW)[NLJIET] Explain for loop with example. (Jan-2020-NEW) [NLJIET]	7 4
	For loop in C A for loop is a more efficient loop structure in 'C' programming. The general structure of for loop syntax in C is as follows: Syntax of For Loop in C: <pre>for (initial value; condition; incrementation or decrementation) { statements; }</pre> • The initial value of the for loop is performed only once.	

- The condition is a Boolean expression that tests and compares the counter to a fixed value after each iteration, stopping the for loop when false is returned.
- The incrementation/decrementation increases (or decreases) the counter by a set value.

Flowchart of for loop:



Following program illustrates the for loop in C programming example:

```
#include<stdio.h>
int main()
{
    int number;
    for(number=1;number<=10;number++) //for loop to print 1-10 numbers
    {
        printf("%d\n",number); //to print the number
    }
    return 0;
}
```

Output:

1

2
3
4
5
6
7
8
9
10

Working of program:

1. We have declared a variable of an int data type to store values.
2. In for loop, in the initialization part, we have assigned value 1 to the variable number. In the condition part, we have specified our condition and then the increment part.
3. In the body of a loop, we have a print function to print the numbers on a new line in the console. We have the value one stored in number, after the first iteration the value will be incremented, and it will become 2. Now the variable number has the value 2. The condition will be rechecked and since the condition is true loop will be executed, and it will print two on the screen. This loop will keep on executing until the value of the variable becomes 10. After that, the loop will be terminated, and a series of 1-10 will be printed on the screen.

In C, the for loop can have multiple expressions separated by commas in each part.

For example:

```
for (x = 0, y = num; x < y; i++, y--) {  
    statements;  
}
```

Nested for loop:

loops can also be nested where there is an outer loop and an inner loop. For each iteration of the outer loop, the inner loop repeats its entire cycle.

Consider the following example with multiple conditions in for loop, that uses nested for loop in C programming to output a multiplication table:

```
#include<stdio.h>
int main() {
int i, j;
int table = 2;
int max = 5;
for (i = 1; i <= table; i++) { // outer loop
    for (j = 0; j <= max; j++) { // inner loop
        printf("%d x %d = %d\n", i, j, i*j);
    }
    printf("\n"); /* blank line between tables */
}}
```

Output:

```
1 x 0 = 0
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
```

```
2 x 0 = 0
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
```

8 Compare for loop and while loop with illustrations. (Jan-2020-OLD) [NLJIET]

3

While Loop in C

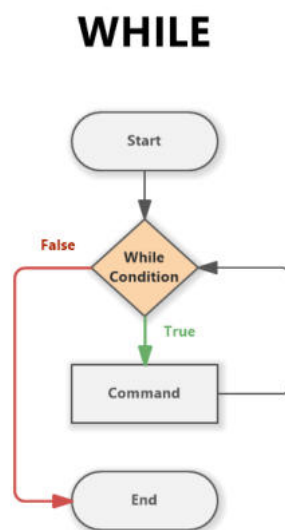
A while loop is the most straightforward looping structure. While loop syntax in C programming language is as follows:

Syntax of While Loop in C:

```
while (condition) {  
    statements;  
}
```

It is an entry-controlled loop. In while loop, a condition is evaluated before processing a body of the loop. If a condition is true then and only then the body of a loop is executed. After the body of a loop is executed then control again goes back at the beginning, and the condition is checked if it is true, the same process is executed until the condition becomes false. Once the condition becomes false, the control goes out of the loop.

Flowchart of while loop:



Following program illustrates while loop in C programming example:

```
#include<stdio.h>  
#include<conio.h>  
int main()  
{
```

```
int num=1;    //initializing the variable
while(num<=10)    //while loop with condition
{
    printf("%d\n",num);
    num++;        //incrementing operation
}
return 0;
}
```

Output:

1
2
3
4
5
6
7
8
9
10

For loop in C

A for loop is a more efficient loop structure in 'C' programming. The general structure of for loop syntax in C is as follows:

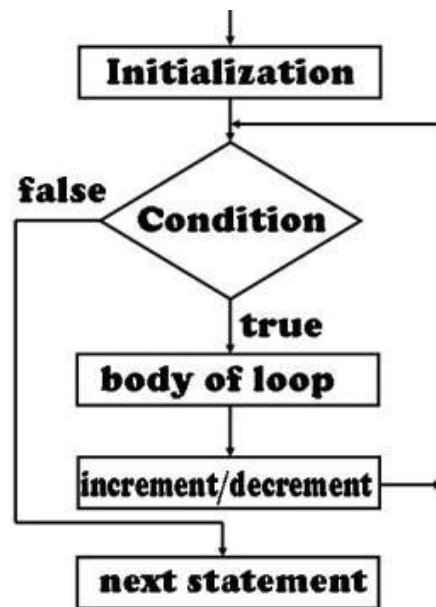
Syntax of For Loop in C:

```
for (initial value; condition; incrementation or decrementation )
{
    statements;
}
```

- The initial value of the for loop is performed only once.
- The condition is a Boolean expression that tests and compares the counter to a fixed value after each iteration, stopping the for loop when false is returned.

- The incrementation/decrementation increases (or decreases) the counter by a set value.

Flowchart of for loop:



Following program illustrates the for loop in C programming example:

```
#include<stdio.h>
int main()
{
    int number;
    for(number=1;number<=10;number++) //for loop to print 1-10 numbers
    {
        printf("%d\n",number);          //to print the number
    }
    return 0;
}
```

Output:

1
2
3



	4 5 6 7 8 9 10	
	PROGRAMS	
1	Write a program to print the following pattern. (Dec-2013-OLD) [NLJIET] 1 0 1 1 0 1 0 1 0 1	7
	<pre>#include<stdio.h> #include<conio.h> void main() { int i,j,n; printf("enter the no of rows"); scanf("%d",&n); for(i=1;i<=n;i++) { for(j=1;j<=i;j++) { printf("%d",((n-i)+j+1)%2); } printf("\n"); }</pre>	
2	Write a program to print the following pattern. (Jun-2014-OLD) [NLJIET] * * * * * * * * * *	7



	<pre> #include<stdio.h> #include<conio.h> void main() { int i,j,k,n; printf("enter the no of rows"); scanf("%d",&n); for(i=1;i<=n;i++) { for(k=1;k<=n-i;k++) { printf(" "); } for(j=1;j<=i;j++) { printf("* "); } printf("\n"); } } </pre>	
3	Write a program to print the following pattern (Jan-2015-OLD) [NLJIET] <pre> 1 2 2 3 3 3 4 4 4 4 </pre>	7
	<pre> #include<stdio.h> #include<conio.h> void main() { int i,j,k,n; printf("enter the no of rows"); scanf("%d",&n); for(i=1;i<=n;i++) { </pre>	



```

    for(k=1;k<=n-i;k++)
    {
        printf(" ");
    }
    for(j=1;j<=i;j++)
    {
        printf("%d ",i);
    }
    printf("\n");
}

```

4 Write a program to print the following pattern. (Dec-2015-OLD) [NLJIET]

7

```

*
* *
* * *
* * * *

```

```

#include<stdio.h>
#include<conio.h>
void main()
{
    int i,j,n;
    printf("enter the no of rows");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=i;j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}

```

5 Write a program to find the sum of first N odd numbers. (Dec-2013-OLD)(Dec-2015-OLD) [NLJIET]

7



```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,j=1,n,sum=0;
    printf("enter value of n: ");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        sum=sum+j;
        j=j+2;
    }
    printf("sum of first %d odd numbers = %d ",n,sum);
}
```

Output:

enter value of n: 5
sum of first 5 odd numbers = 25

6 Write a program in 'C' to print the following pattern. (Jun-2017-OLD)
[NLJIET]

1
23
456
78910

7

॥ विद्या ऽमृतम् ॥

Exploring Emerging Technologies

Program for n=4:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,j,n,k=1;
    printf("enter the no of rows");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=i;j++)
        {
            printf("%d ",k);
            k++;
        }
        printf("\n");
    }
}
```

7

6.1 Write a program in 'C' to print the following pattern. (Jan-2019-OLD)
[NLJIET]

```
12345
2345
345
45
5
```

Exploring Emerging Technologies

Program for n=5

```
#include<stdio.h>
#include<conio.h>
void main()
```

```
{
    int i,j,n;
    printf("enter the no of rows");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        for(j=i;j<=n;j++)
        {
            printf("%d ",j);
        }
        printf("\n");
    }
}
```

7

6.2 Write a C program to print following pattern. (Mar-2022-NEW) [NLJIET]

```
1
2 3
4 5 6
```

Program for n=3:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,j,n,k=1;
    printf("enter the no of rows");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=i;j++)
        {
            printf("%d ",k);
            k++;
        }
        printf("\n");
    }
```

Exploring Emerging Technologies

	<pre> } } </pre>	
7	Write a program in 'C' to print the following pattern using loop statement. (May-2018- OLD) (Jan-2020-NEW) [NLJIET]	7
	<pre> 1 2 2 3 3 3 4 4 4 4 5 5 5 5 5 </pre>	
	Program for n=5: <pre> #include<stdio.h> #include<conio.h> void main() { int i,j,n; printf("enter the no of rows"); scanf("%d",&n); for(i=1;i<=n;i++) { for(j=1;j<=i;j++) { printf("%d ",i); } printf("\n"); } } </pre> Exploring Emerging Technologies Write a C program to display given pattern. <pre> 1 12 1 2 3 </pre>	7

```
1 2 3 4
1 2 3 4 5
```

Program for n=5:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,j,n;
    printf("enter the no of rows");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=i;j++)
        {
            printf("%d ",j);
        }
        printf("\n");
    }
}
```

8 Write a C program to print following pattern. (Mar-2022-NEW) [NLJIET]

```
1
2 2
3 3 3
```

7

Program for n=3:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,j,k,n;
```



```

printf("enter the no of rows");
scanf("%d",&n);
for(i=1;i<=n;i++)
{
    for(k=1;k<=n-i;k++)
    {
        printf(" ");
    }
    for(j=1;j<=i;j++)
    {
        printf("%d ",i);
    }
    printf("\n");
}
}

```

9

Write a program that reads a value n and generates the following pattern: (Jan-2020- OLD) [NLJIET]

7

```

      1
     1 2 3
    1 2 3 4 5
   1 2 3 4 5 6 7
  1 2 3 4 5 6 7 8 9

```

Program for n=5:

```

#include<stdio.h>
#include<conio.h>
void main()
{
    int i,j,k,n;
    printf("enter the no of rows");

```



```
scanf("%d",&n);
for(i=1;i<=n;i++)
{
    for(k=1;k<=n-i;k++)
    {
        printf(" ");
    }
    for(j=1;j<=2*i-1;j++)
    {
        printf("%d ",j);
    }
    printf("\n");
}
}
```

10 Write a program to print the triangle shown below. (Aug-2022-NEW)[NLJIET]

4

A
A B
A B C
A B C D

Program for n=4:

```
#include<stdio.h>
#include<conio.h>
void main()
```

```
{
    int i,j,n;
    printf("enter the no of rows");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=i;j++)
        {
            printf("%c ",64+j);
        }
    }
}
```



	<pre> printf("\n"); } } </pre>	
11	Write a C program to find $1 + 1/2! + 1/3! + 1/4! + \dots + 1/n!$. (Nov-2020-NEW) [NLJIET]	7
	<pre> #include<stdio.h> #include<conio.h> void main() { int i,n; float f=1,sum=0,inv; printf("enter the no of terms n:"); scanf("%d",&n); for(i=1;i<=n;i++) { f=f*i; inv=1/f; sum=sum+inv; } printf("\n sum is %f",sum); } </pre> Output: enter the no of terms n:4 sum is 1.708333	
12	Write a C Program to check whether the given number is prime or not. (Jan-2015-OLD) [NLJIET]	4
	<pre> #include<stdio.h> #include<conio.h> void main() { int i,n,count=0; </pre>	



	<pre> printf("enter the no to check prime: "); scanf("%d",&n); for(i=1;i<=n;i++) { if(n%i==0) { count++; } } if(count==2) { printf("Number is prime"); } else { printf("Number is not prime"); } } </pre>	
	Output: enter the no to check prime: 5 Number is prime	
13	Write a program to find sum of first N odd numbers. Ex. 1+3+5+7+.....+N. (Jan-2017-OLD) (Jan-2020-NEW) [NLJIET]	7
	<pre> #include<stdio.h> #include<conio.h> void main() { int i,j=1,n,sum=0; printf("enter value of n: "); scanf("%d",&n); for(i=1;i<=n;i++) { sum=sum+j; j=j+2; } } </pre>	



	<pre>printf("sum of first %d odd numbers = %d ",n,sum); }</pre> Output: enter value of n: 5 sum of first 5 odd numbers = 25	
14	Write a C program to find factorial of a given number. (Nov-2020-NEW) (Mar-2023-NEW)[NLJIET]	7
	<pre>#include<stdio.h> #include<conio.h> void main() { int i,f=1,n; printf("Enter the no to find factorial: "); scanf("%d",&n); for(i=1;i<=n;i++) { f=f*i; } printf("The factorial of %d is %d",n,f); }</pre> Output: Enter the no to find factorial: 5 The factorial of 5 is 120	
15	Write a program to add first n numbers. Get value of n from user. (Jun-2019-OLD) [NLJIET]	3
	<pre>#include<stdio.h> #include<conio.h> void main() { int i,n,sum=0;</pre>	



```
printf("enter value of n: ");
scanf("%d",&n);
for(i=1;i<=n;i++)
{
    sum=sum+i;
}
printf("sum of first %d numbers = %d ",n,sum);
}
```

Output:

enter value of n: 10
sum of first 10 numbers = 55

16 Write a program to reverse a given number. **.(Jun-2019-NEW) [NLJIET]**

4

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int num,rev=0,dig,x;
    printf("enter the number: ");
    scanf("%d",&num);
    x=num;
    while(num!=0)
    {
        dig=num%10;
        rev=(rev*10)+dig;
        num=num/10;
    }
    printf("The reverse of number %d is %d",x,rev);
}
```

Output:

enter the number: 157
The reverse of number 157 is 751



17	Write a program to find $1+1/2+1/3+1/4+\dots+1/n$. .(Jun-2019-NEW) [NLJIET]	4
	<pre> #include<stdio.h> #include<conio.h> void main() { int n; float i,sum=0; printf("enter value of n: "); scanf("%d",&n); for(i=1;i<=n;i++) { sum=sum+1/i; } printf("sum of first %d numbers = %f ",n,sum); } </pre> Output: enter value of n: 3 sum of first 3 numbers = 1.833333	
18	Write a program to print all Armstrong numbers in a given range. Armstrong number is equal to sum of cubes of its individual digits. For example $153 = 1^3 + 5^3 + 3^3$. So, 153 is Armstrong number. .(Jun-2019-NEW) [NLJIET]	7
	<pre> #include<stdio.h> #include<conio.h> void main() { int s,e,sum=0,dig,x,i,y; printf("enter the starting and ending range: "); scanf("%d %d",&s,&e); for(i=s;i<=e;i++) { x=i; y=i; </pre>	


```

sum=0;
while(y!=0)
{
    dig=y%10;
    sum=sum+(dig*dig*dig);
    y=y/10;
}
if(x==sum)
    printf("\n %d",x);
}

```

Output:

enter the starting and ending range: 100
 999
 Armstrong number in given range are :
 153
 370
 371
 407

19 Write a program to print Fibonacci series.e.g.0, 1, 1, 2, 3, 5, 8, 13.(Aug-2022-NEW)[NLJIET]

4

```

#include<stdio.h>
#include<conio.h>
void main()
{
    int a=0,b=1,sum=0,i,n;
    printf("enter the no of terms for fibonacci series ");
    scanf("%d",&n);
    printf("The fibonacci series : ");
    printf("%d\t%d",a,b);
    for(i=3;i<=n;i++)
    {
        sum=a+b;
        printf("\t %d",sum);
    }
}

```




	<pre> a=b; b=sum; } } </pre> Output: enter the no of terms for fibonacci series 10 The fibonacci series : 0 1 1 2 3 5 8 13 21 34	
20	Write a C program to make sum of digits of a given number. (if input is 145, output should be 10).(Mar-2023-NEW)[NLJIET]	7
	<pre> #include<stdio.h> #include<conio.h> void main() { int s,sum=0,dig,y; printf("enter the number: "); scanf("%d",&s); y=s; while(y!=0) { dig=y%10; sum=sum+dig; y=y/10; } printf("The sum of digit of %d is %d",s,sum); } </pre> Output: enter the number: 25698 The sum of digit of 25698 is 30	
21	Examine following code and give output of it.(Mar-2023-NEW)[NLJIET]	4
	<pre> void main(){ int i=1; </pre>	



	<pre>for (i=10; i>=1 ; --i) { printf (“ %d”, i); i--; } }</pre>	
	Explanation: 1) i=10, i>=1 condition is true, printf statement will print 10 , i- - in loop body makes it 9 and - - i of for loop syntax makes it 8 2) i=8, i>=1 condition is true, printf statement will print 8 , i- - in loop body makes it 7 and - - i of for loop syntax makes it 6 3) i=6, i>=1 condition is true, printf statement will print 6 , i- - in loop body makes it 5 and - - i of for loop syntax makes it 4 4) i=4, i>=1 condition is true, printf statement will print 4 , i- - in loop body makes it 3 and - - i of for loop syntax makes it 2 5) i=2, i>=1 condition is true, printf statement will print 8 , i- - in loop body makes it 1 and - - i of for loop syntax makes it 0 6) i=0, i>=1 condition is false so loop terminates. Output: 10 8 6 4 2	
22	Find out error(s), if any in the following code and correct it: (Mar-2023-NEW)[NLJIET] <pre>int main() { printf(“%f%d”, 7.0%5.0, 5 -2); }</pre>	3
	Error: error: invalid operands to binary % (have 'double' and 'double') Corrected code: <pre>#include<stdio.h></pre>	



	<pre>#include<conio.h> int main() { printf("%d %d",7%5,5 -2); }</pre> Output: 2 1 Explanation: The modulo operator, denoted by %, is an arithmetic operator. The modulo division operator produces the remainder of an integer division which is also called the modulus of the operation. The % operator cannot be applied to floating-point numbers i.e float or double. If you try to use the modulo operator with floating-point constants or variables, the compiler will produce an error.	
23	Examine following code and give output of it.(Mar-2023-NEW)[NLJIET] <pre>void main(){ int a=0; while (a<10) { a++; if (a%3==1) { continue; } printf (" %d", a); }}</pre>	4
	Output: 2 3 5 6 8 9 Explanation: a=0, a<10 condition true, a++ gives a=1 , a%3=1%3=1 hence continue a=1, a<10 condition true, a++ gives a=2 , a%3=2%3=2 hence print 2 a=2, a<10 condition true, a++ gives a=3 , a%3=3%3=0 hence print 3 a=3, a<10 condition true, a++ gives a=4 , a%3=4%3=1 hence continue a=4, a<10 condition true, a++ gives a=5 , a%3=5%3=2 hence print 5	



	a=5, a<10 condition true, a++ gives a=6 , a%3=6%3=0 hence print 6 a=6, a<10 condition true, a++ gives a=7 , a%3=7%3=1 hence continue a=7, a<10 condition true, a++ gives a=8 , a%3=8%3=2 hence print 8 a=8, a<10 condition true, a++ gives a=9 , a%3=9%3=0 hence print 9 a=9, a<10 condition true, a++ gives a=10, a%3=10%3=1 hence continue a=10, a<10 condition false, so execution stops. Output: 2 3 5 6 8 9	
24	Find out error(s), if any in the following code and correct it: (Mar-2023-NEW)[NLJIET] <pre>int main() { if(10%2==0){ break; } }</pre>	3
	Error: break statement not within loop or switch Corrected code: <pre>int main() { while(10%2){ break; } }</pre>	
25	Write a C program to print following pattern: (Mar-2023-NEW)[NLJIET] <pre>1 2 2 3 3 3 4 4 4 4</pre>	7
	<pre>#include<stdio.h> #include<conio.h> void main()</pre>	



	<pre> { int i,j,n; printf("enter the no of rows"); scanf("%d",&n); for(i=1;i<=n;i++) { for(j=1;j<=i;j++) { printf("%d ",i); } printf("\n"); } } </pre>	
26	Write a C program to check whether the entered character is alphabet, digit, space or any special character. (Mar-2023-NEW)[NLJIET]	7
	<pre> #include<stdio.h> #include<conio.h> void main() { char c; printf("Enter character"); scanf("%c",&c); if(c>='a' && c<='z' c>='A' && c<='Z') printf("It is an alphabet"); else if (c>='0' && c<='9') printf("It is a number"); else if (c==' ') printf("It is a blank space"); else printf("It is a special character"); } </pre> Output: Enter character: 6	



	It is a number	
27	Write a program which takes 20 numbers from user and display maximum, minimum and average of those numbers.(Mar-2023-OLD)[NLJIET]	4
	Program of Array – Solution will be given in chapter 4	
28	Write a program which takes the amount of bill and the percentage of GST to be charged on the bill amount from user and display total amount payable.(Mar-2023-OLD)[NLJIET]	7
	Program: <pre>#include<stdio.h> #include<conio.h> void main() { double price; int gst; double amount; printf("Enter the bill amount\n"); //original price of the item scanf("%lf",&price); printf("Enter the rate of GST\n"); //rate of tax on the item scanf("%d",&gst); amount=price+(gst*price)/100; //total amount after adding GST printf("The total payable amount is %lf",amount); }</pre> Output: Enter the bill amount 2000 Enter the rate of GST 5 The total payable amount is 2100.000000	
	TOPIC: 4 Break, Continue and Goto Statement	
	SHORT QUESTIONS	
1	Give usage of break and continue. [NLJIET]	2
	<u>Break statement:</u>	

The **break** statement in C programming has the following two usages –

- When a **break** statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.
- It can be used to terminate a case in the **switch** statement (covered in the next chapter).

If you are using nested loops, the break statement will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax

The syntax for a **break** statement in C is as follows –

break;

Continue Statement:

The **continue** statement in C language is used to bring the program control to the beginning of the loop. The continue statement skips some lines of code inside the loop and continues with the next iteration. It is mainly used for a condition so that we can skip some code for a particular condition.

Syntax:

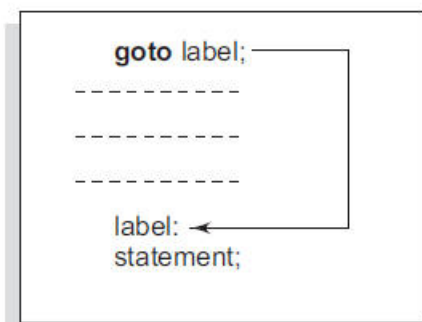
//loop statements

continue;

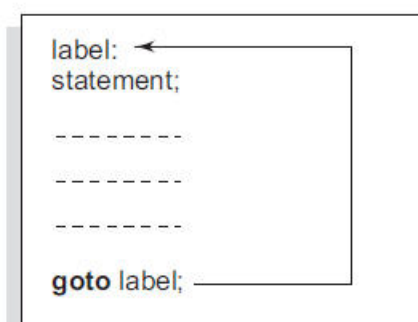
//some lines of the code which is to be skipped

Exploring Emerging Technologies

	The break statement in C programming has the following two usages – • When a break statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop. • It can be used to terminate a case in the switch statement (covered in the next chapter). If you are using nested loops, the break statement will stop the execution of the innermost loop and start executing the next line of code after the block. Syntax The syntax for a break statement in C is as follows – break; Continue Statement: The continue statement in C language is used to bring the program control to the beginning of the loop. The continue statement skips some lines of code inside the loop and continues with the next iteration. It is mainly used for a condition so that we can skip some code for a particular condition. Syntax: //loop statements continue; //some lines of the code which is to be skipped Exploring Emerging Technologies	
2	Give syntax of goto statement. [NLJIET]	1
	C supports the goto statement to branch unconditionally from one point to another in the program. The syntax is as shown below.	



Forward jump



Backward jump

Example:

```
void main()
{
double x,y;
printf("enter no: ");
scanf("%lf",&x);
if (x > 0)
    goto read;
read:
y = sqrt(x);
printf("%lf%lf\n",x,y);
return;
}
```

Output:

enter no: 36
36.000000 6.000000

DESCRIPTIVE QUESTIONS

1	Discuss general form of following decision – making statements. (I) IF (II) SWITCH (III) GOTO. (Jan-2017-OLD) [NLJIET] Define general form of 1) do while loop 2) Nested if 3) goto. (Jan-2019-NEW) [NLJIET] Explain the use of following statements. (I) if...else (III) goto (Jan-2019-OLD) [NLJIET]	3
		3
		4
(I)	IF	

The if Statement

Use the if statement to specify a block of code to be executed if a condition is true.

SYNTAX:

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

Example

```
void main()  
{  
    int x = 20;  
    int y = 18;  
    if (x > y) {  
        printf("x is greater than y");  
    }  
}
```

Output:

x is greater than y

(II) SWITCH

In C, the switch case statement is used for executing one condition from multiple conditions. It is similar to an if-else-if ladder.

Syntax of switch Statement in C

```
switch(expression)  
{  
    case value1: statement_1;
```

```
break;
case value2: statement_2;
break;
.
.
.
case value_n: statement_n;
break;

default: default_statement;
}
```

Example:

```
#include <stdio.h>

int main()
{
    // switch variable
    int var = 1;

    // switch statement
    switch (var) {
        case 1:
            printf("Case 1 is Matched.");
            break;

        case 2:
            printf("Case 2 is Matched.");
            break;

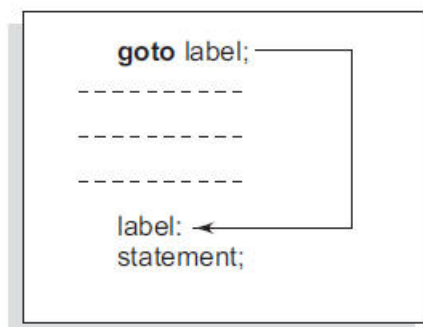
        case 3:
            printf("Case 3 is Matched.");
            break;
```

```
default:
    printf("Default case is Matched.");
    break;
}
```

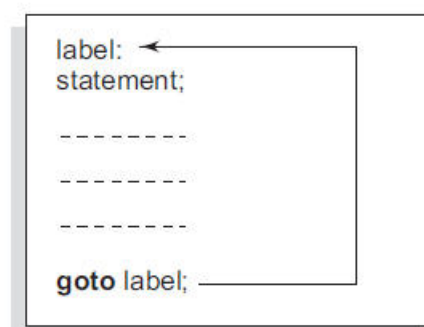
```
return 0;
}
```

(III) Goto statement

C supports the goto statement to branch unconditionally from one point to another in the program. The syntax is as shown below.



Forward jump



Backward jump

Example:

```
void main()
{
    double x,y;
    printf("enter no: ");
    scanf("%lf",&x);
    if (x > 0)
        goto read;
    read:
    y = sqrt(x);
    printf("%lf%lf\n",x,y);
    return;
}
```

Output:



enter no: 36

36.000000 6.000000

(IV) If...else

Syntax:

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

Example:

```
int time = 20;  
if (time < 18) {  
    printf("Good day.");  
} else {  
    printf("Good evening.");  
}  
// Output "Good evening."
```

(V) Nested if

It is always legal in C programming to **nest** if-else statements, which means you can use one if or else if statement inside another if or else if statement(s).

Syntax

The syntax for a nested if statement is as follows –

```
if( boolean_expression 1 ) {  
  
    /* Executes when the boolean expression 1 is true */  
    if(boolean_expression 2) {  
        /* Executes when the boolean expression 2 is true */  
    }  
}
```

```
}
```

Example

```
#include <stdio.h>
```

```
int main () {
```

```
    /* local variable definition */
```

```
    int a = 100;
```

```
    int b = 200;
```

```
    /* check the boolean condition */
```

```
    if( a == 100 ) {
```

```
        /* if condition is true then check the following */
```

```
        if( b == 200 ) {
```

```
            /* if condition is true then print the following */
```

```
            printf("Value of a is 100 and b is 200\n" );
```

```
        }
```

```
    }
```

```
    printf("Exact value of a is : %d\n", a );
```

```
    printf("Exact value of b is : %d\n", b );
```

```
    return 0;
```

```
}
```

Output:

Value of a is 100 and b is 200

Exact value of a is : 100

Exact value of b is : 200

(VI) Do...while

A do...while loop in C is similar to the while loop except that the condition is always executed after the body of a loop. It is also called an exit-controlled loop. In the do-while loop, the body of a loop is always executed at least once. After the body is executed, then it checks the condition. If the condition is true, then it will again execute the body of a loop otherwise control is transferred out of the loop.

Syntax of Do-While Loop in C:

```
do {
    statements
} while (expression);
```

Example:

```
void main()
```

```
{
```

```
int i=0;
```

```
do
```

```
{
```

```
    printf("Hello");
```

```
} while(i!=0);
```

```
}
```

Output:

Hello

- | | |
|--|-----------------|
| <p>2 Explain: 1. Nesting if-else statement 2. Use of break statement. (May-2018-OLD)[NLJIET]</p> | <p>4</p> |
| <p>Explain 1) nested if-else statement 2) break statement. (Nov-2020-NEW)[NLJIET]</p> | <p>4</p> |

Nesting if-else statement:



Nested "if else statements" simply means the use of conditional statement if..else inside another conditional statement if..else.

The basic format of the Nested if-else statement is:

Syntax:

```
if(test_expression one)
{
    //Statement block Executes when the test expression one is true.
    if(test_expression two) {
        //Statement block Executes when the test expression two is true.
    }
    else
    {
        //else statement block- Statement block Executes when the test expression
        two is false
    }
}
else
{
    //else statement block - Statement block Executes when the test expression one
    is false
}
```

Example:

program to select and print the largest of the three nos. using nested-if-else statement

```
#include <stdio.h>
#include <conio.h>
```

```
int main() {
```

```
    int n1, n2, n3;
```

```
    printf("Enter three numbers: ");
```

```
scanf("%d %d %d", &n1, &n2, &n3);
```

```
// outer if statement
```

```
if (n1 >= n2) {
```

```
    // inner if...else
```

```
    if (n1 >= n3)
```

```
        printf("%d is the largest number.", n1);
```

```
    else
```

```
        printf("%d is the largest number.", n3);
```

```
    }
```

```
// outer else statement
```

```
else {
```

```
    // inner if...else
```

```
    if (n2 >= n3)
```

```
        printf("%d is the largest number.", n2);
```

```
    else
```

```
        printf("%d is the largest number.", n3);
```

```
    }
```

```
return 0;
```

```
}
```

Output:

Enter three numbers: 2

4

6

Exploring Emerging Technologies

6 is the largest number.

Break statement:

The **break** statement in C programming has the following two usages –

- When a **break** statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.
- It can be used to terminate a case in the **switch** statement (covered in the next chapter).

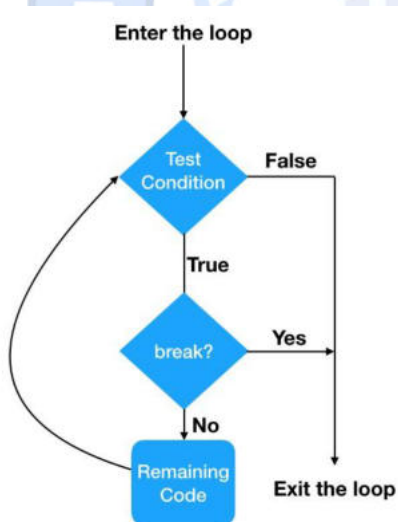
If you are using nested loops, the break statement will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax

The syntax for a **break** statement in C is as follows –

break;

Flow Diagram



Example

```
#include <stdio.h>
```

```
int main () {
```

```
    /* local variable definition */
```



```
int a = 10;
```

```
/* while loop execution */
```

```
while( a < 20 ) {
```

```
    printf("value of a: %d\n", a);
```

```
    a++;
```

```
    if( a > 15) {
```

```
        /* terminate the loop using break statement */
```

```
        break;
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```

When the above code is compiled and executed, it produces the following result

–

value of a: 10

value of a: 11

value of a: 12

value of a: 13

value of a: 14

value of a: 15

3

Explain break and continue statement with example. (Jun-2016-OLD) (Jan-2019-OLD) (Jan-2019-NEW) [NLJIET]

Describe break and continue statement with suitable example.(Jun-2019-OLD) [NLJIET]

Define break and continue statement with example. (Jan-2020-NEW) [NLJIET]

Discuss use of break and continue statement in C with example.(Mar-2022-NEW) [NLJIET]

Discuss the need of break and continue statements with example.(Aug-2022-

4

4

3

3

NEW)[NLJIET]

4

Break statement:

The **break** statement in C programming has the following two usages –

- When a **break** statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.
- It can be used to terminate a case in the **switch** statement (covered in the next chapter).

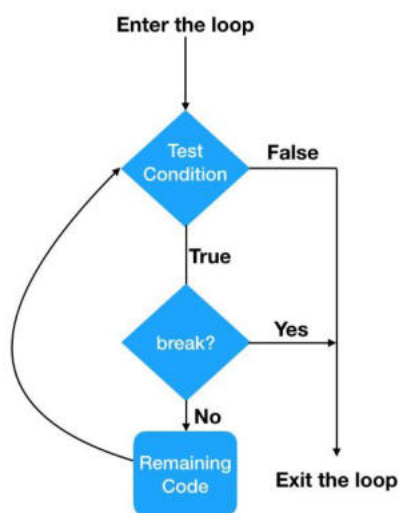
If you are using nested loops, the break statement will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax

The syntax for a **break** statement in C is as follows –

```
break;
```

Flow Diagram



Example

```
#include <stdio.h>
```

```
int main () {  
  
    /* local variable definition */  
    int a = 10;  
  
    /* while loop execution */  
    while( a < 20 ) {  
  
        printf("value of a: %d\n", a);  
        a++;  
  
        if( a > 15) {  
            /* terminate the loop using break statement */  
            break;  
        }  
    }  
  
    return 0;  
}
```

When the above code is compiled and executed, it produces the following result
—

```
value of a: 10  
value of a: 11  
value of a: 12  
value of a: 13  
value of a: 14  
value of a: 15
```

Continue Statement:

The **continue statement** in C language is used to bring the program control to the beginning of the loop. The continue statement skips some lines of code inside the loop and continues with the next iteration. It is mainly used for a condition so that we can skip some code for a particular condition.

Syntax:

```
//loop statements
```

```
continue;
```

```
//some lines of the code which is to be skipped
```

Continue statement example

```
#include<stdio.h>
```

```
int main(){
```

```
int i=1;//initializing a local variable
```

```
//starting a loop from 1 to 10
```

```
for(i=1;i<=10;i++){
```

```
if(i==5){//if value of i is equal to 5, it will continue the loop
```

```
continue;
```

```
}
```

```
printf("%d \n",i);
```

```
//end of for loop
```

```
return 0;
```

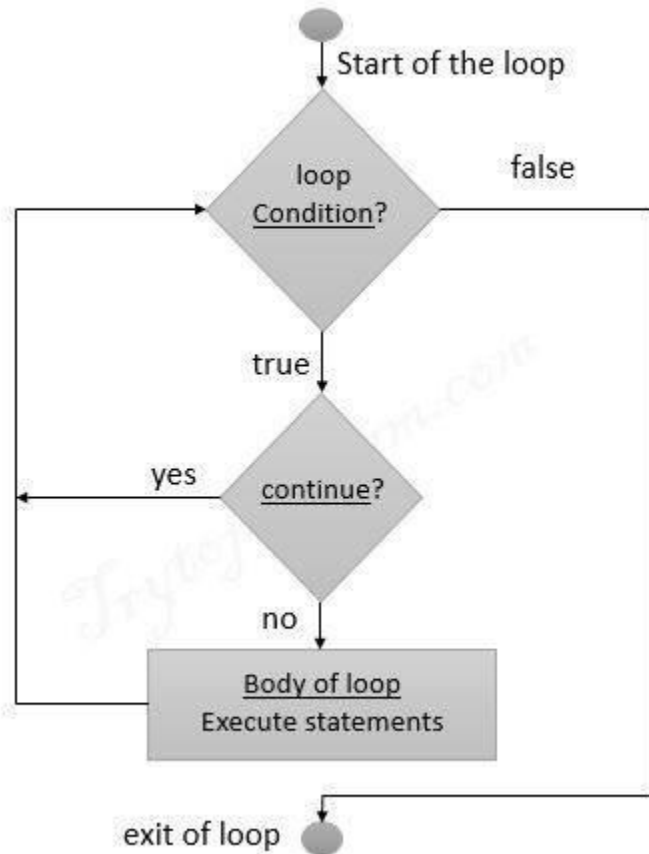
```
}
```

Output

```
1
2
3
4
6
7
8
9
10
```

5 is not printed on the console because loop is continued at i==5.

Flow Diagram



4

Compare and contrast goto statement and switch statement with example.(Aug-2022-NEW)[NLJIET]

4

The **C goto statement** is a jump statement which is sometimes also referred to as an **unconditional jump** statement. The goto statement can be used to jump from anywhere to anywhere within a function.

Syntax:

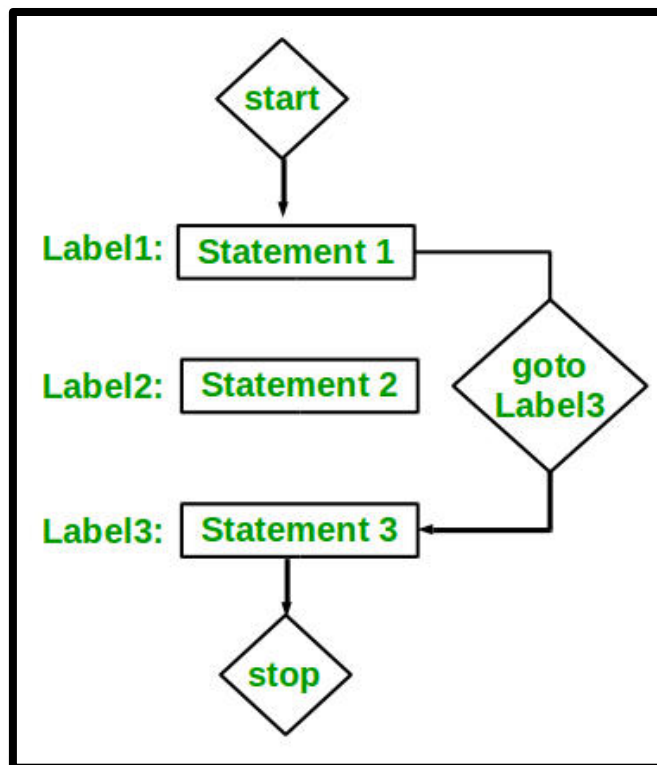
Syntax1 | Syntax2

goto label; | label:

. | .
. | .

label: | goto label;

In the above syntax, the first line tells the compiler to go to or jump to the statement marked as a label. Here, the label is a user-defined identifier that indicates the target statement. The statement immediately followed after 'label:' is the destination statement. The 'label:' can also appear before the 'goto label;' statement in the above syntax.



Example of goto statement

```
#include <stdio.h>
int main()
{
    int sum=0;
    for(int i = 0; i<=10; i++){
        sum = sum+i;
        if(i==5){
            goto addition;
        }
    }
}
```

```
}  
  
addition:  
printf("%d", sum);  
  
return 0;  
}
```

Output:

15

Explanation: In this example, we have a label addition and when the value of i (inside loop) is equal to 5 then we are jumping to this label using goto. This is reason the sum is displaying the sum of numbers till 5 even though the loop is set to run from 0 to 10.

Switch statement:

In C, the switch case statement is used for executing one condition from multiple conditions. It is similar to an if-else-if ladder.

The switch statement consists of conditional-based cases and a default case.

Syntax of switch Statement in C

```
switch(expression)  
{  
case value1: statement_1;  
    break;  
case value2: statement_2;  
    break;  
.  
.  
.  
case value_n: statement_n;  
    break;
```

```
default: default_statement;  
}
```

Rules for switch statement in C language

- 1) The *switch expression* must be of an integer or character type.
- 2) The *case value* must be an integer or character constant.
- 3) The *case value* can be used only inside the switch statement.
- 4) The *break statement* in switch case is not must. It is optional. If there is no break statement found in the case, all the cases will be executed present after the matched case. It is known as *fall through* the state of C switch statement.

Break in switch case

This keyword is used to stop the execution inside a switch block. It helps to terminate the switch block and break out of it. When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.

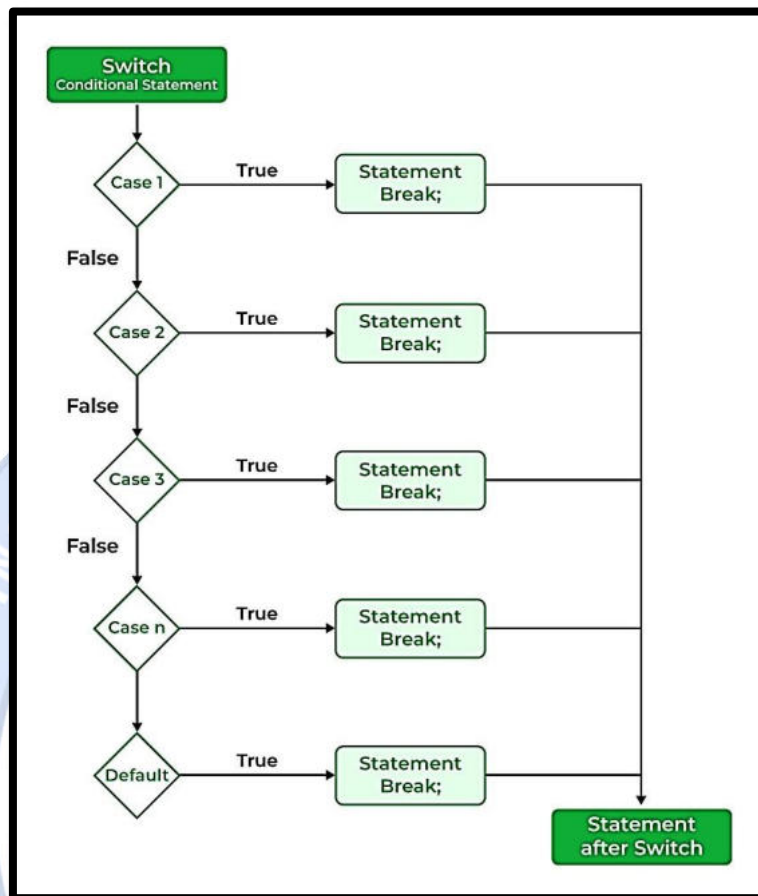
The **break statement is optional**. If omitted, execution will continue on into the next case. The flow of control will fall through to subsequent cases until a break is reached.

Default in switch case

The default keyword is used to specify the set of **statements to execute if there is no case match**.

It is **optional** to use the default keyword in a switch case. Even if the switch case statement does not have a default statement, it would run without any problem.

Flowchart of switch statement:



Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
// switch variable
```

```
int var = 1;
```

```
// switch statement
```

```
switch (var) {
```

```
case 1:
```

```
printf("Case 1 is Matched.");
```

```
break;
```

```
case 2:
```

```

printf("Case 2 is Matched.");
break;
case 3:
printf("Case 3 is Matched.");
break;

default:
printf("Default case is Matched.");
break;
}

return 0;
}

```

Output

Case 1 is Matched.

How switch Statement Work?

The working of the switch statement in C is as follows:

1. **Step 1:** The switch variable is evaluated.
2. **Step 2:** The evaluated value is matched against all the present cases.
3. **Step 3A:** If the matching case value is found, the associated code is executed.
4. **Step 3B:** If the matching code is not found, then the default case is executed if present.
5. **Step 4A:** If the break keyword is present in the case, then program control breaks out of the switch statement.
6. **Step 4B:** If the break keyword is not present, then all the cases after the matching case are executed.
7. **Step 5:** Statements after the switch statement are executed.

Exploring Emerging Technologies

5	Explain the use of switch keyword with proper example.(Mar-2023-OLD)[NLJIET]	4
	<u>Switch statement:</u>	

In C, the switch case statement is used for executing one condition from multiple conditions. It is similar to an if-else-if ladder.

The switch statement consists of conditional-based cases and a default case.

Syntax of switch Statement in C

```
switch(expression)
{
case value1: statement_1;
    break;
case value2: statement_2;
    break;
.
.
.
case value_n: statement_n;
    break;

default: default_statement;
}
```

Rules for switch statement in C language

- 1) The *switch expression* must be of an integer or character type.
- 2) The *case value* must be an integer or character constant.
- 3) The *case value* can be used only inside the switch statement.
- 4) The *break statement* in switch case is not must. It is optional. If there is no break statement found in the case, all the cases will be executed present after the matched case. It is known as *fall through* the state of C switch statement.

Break in switch case

This keyword is used to stop the execution inside a switch block. It helps to terminate the switch block and break out of it. When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.

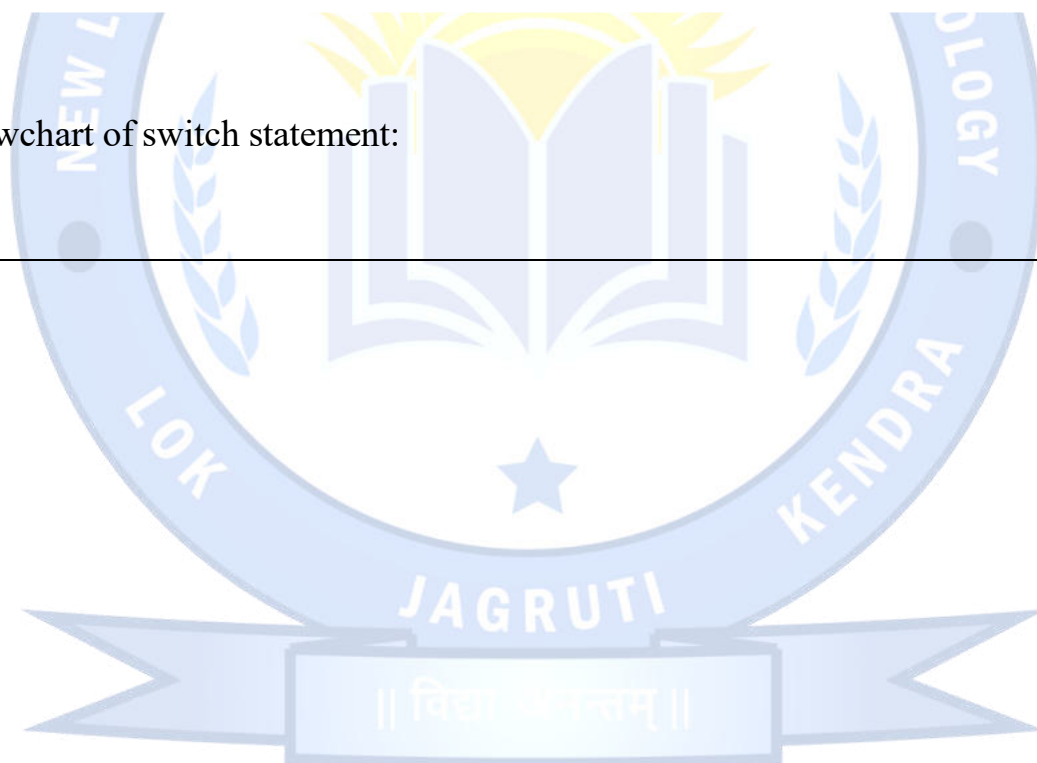
The **break statement is optional**. If omitted, execution will continue on into the next case. The flow of control will fall through to subsequent cases until a break is reached.

Default in switch case

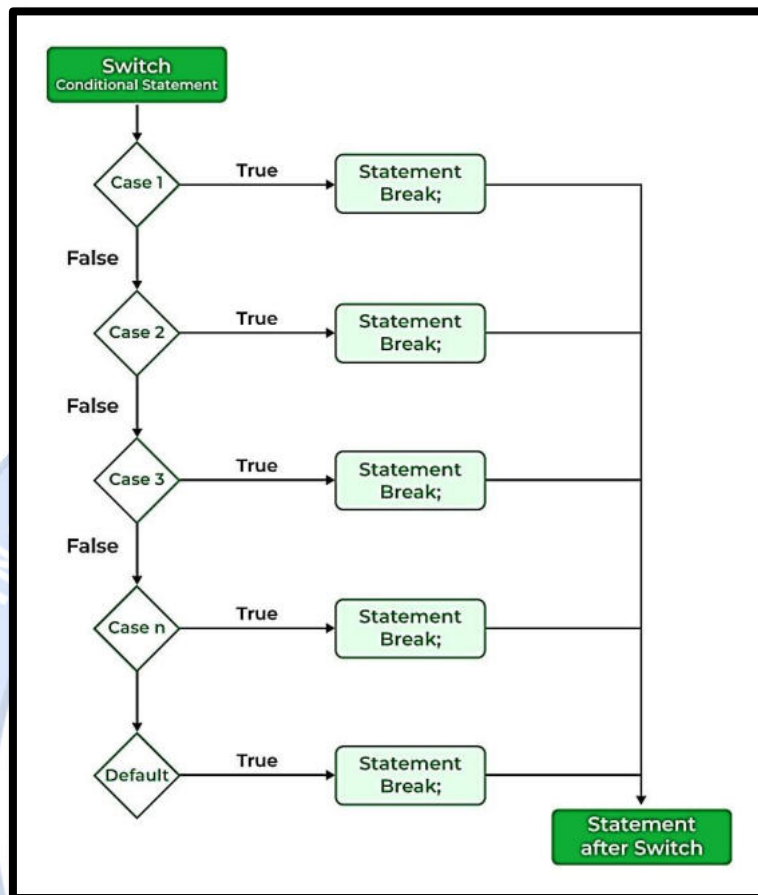
The default keyword is used to specify the set of **statements to execute if there is no case match**.

It is **optional** to use the default keyword in a switch case. Even if the switch case statement does not have a default statement, it would run without any problem.

Flowchart of switch statement:



Exploring Emerging Technologies



Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
// switch variable
```

```
int var = 1;
```

```
// switch statement
```

```
switch (var) {
```

```
case 1:
```

```
printf("Case 1 is Matched.");
```

```
break;
```

```
case 2:
```

```
printf("Case 2 is Matched.");  
break;  
  
case 3:  
    printf("Case 3 is Matched.");  
    break;  
  
default:  
    printf("Default case is Matched.");  
    break;  
}  
  
return 0;  
}
```

Output

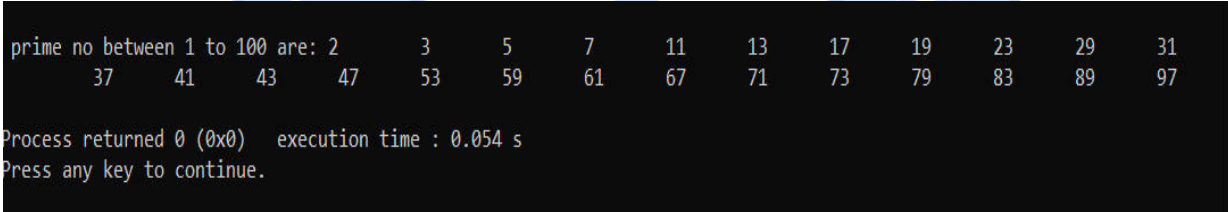
Case 1 is Matched.

How switch Statement Work?

The working of the switch statement in C is as follows:

- **Step 1:** The switch variable is evaluated.
- **Step 2:** The evaluated value is matched against all the present cases.
- **Step 3A:** If the matching case value is found, the associated code is executed.
- **Step 3B:** If the matching code is not found, then the default case is executed if present.
- **Step 4A:** If the break keyword is present in the case, then program control breaks out of the switch statement.
- **Step 4B:** If the break keyword is not present, then all the cases after the matching case are executed.
- **Step 5:** Statements after the switch statement are executed.

PROGRAMS

1	Write a C program to display prime number between 1 to 100. (Jun-2016-OLD) [NLJIET]	7
	<pre> #include<stdio.h> #include<conio.h> void main() { int i,j,count=0; printf("\n prime no between 1 to 100 are:"); for(i=1;i<=100;i++) { count=0; for(j=1;j<=i;j++) { if(i%j==0) count++; } if(count==2) printf(" %d\t",i); } } </pre> Output:  <pre> prime no between 1 to 100 are: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 Process returned 0 (0x0) execution time : 0.054 s Press any key to continue. </pre>	

Exploring Emerging Technologies